CS683 – Natural Language Processing

Course Project Report

Knowing When Not to Answer

Abstention Drift in Multi-Turn Retrieval-Augmented Conversations

Team Members

Member 1 Member 2 Member 3 Member 4 Member 5

Submitted to: *Course Instructor, CS683*

M.Tech (Computer Science and Engineering)

October 2026

Abstract

Chat assistants built on retrieval-augmented generation (RAG) answer questions using documents they retrieve from a collection. When those documents do not contain the answer, the assistant should say "I don't know" (*abstain*) instead of guessing. Most studies of this behaviour look at single questions, but real assistants hold long conversations.

This project asked two research questions. **RQ1:** does the assistant get worse at recognising unanswerable questions as a conversation goes on (*abstention drift*)? **RQ2:** can a small answerability checker, trained on unanswerable examples that we generate cheaply by removing the supporting documents, improve abstention without refusing more answerable questions?

We built a complete multi-turn RAG pipeline that runs on a 4 GB laptop GPU. It uses the **Qwen2.5-3B-Instruct** large language model for query rewriting, answer generation and data filtering; **BM25** and the **BGE-small** embedding model for retrieval; the **BGE re-ranker** for re-ranking and as the answerability checker; and **DeBERTa-v3-base** for earlier checker versions. We evaluated six systems on the SemEval-2026 MTRAG-UN benchmark (429 turns, 97 unanswerable) and ran a controlled probe of 1,221 examples.

Results: (1) there is **no abstention drift**: recall on unanswerable questions stays flat across turns, and we can rule out declines larger than about one percentage point per turn; (2) 23% of the cheaply generated "unanswerable" examples were actually still answerable and had to be filtered out; (3) no answerability checker improved the overall trade-off, as each one added more wrong refusals (8–13) than correct abstentions (3–8); (4) the main weakness of the pipeline is **over-refusal**: it refuses 18–28% of questions that can be answered. Finally, we extended the pipeline with a **LangGraph agent** (built with LangChain) that searches again before refusing, asks clarifying questions and double-checks doubtful answers. Retrying before refusing cut wrong refusals from 18.4% to 13.6%, and adding a self-check calibrated on development data gave the best F1 of all systems (.511 vs .443 for the fixed pipeline). We then deployed the agent as a production web service (FastAPI, PostgreSQL, Qdrant, Celery, Guardrails AI, Langfuse, Prometheus and Grafana, with an automated quality gate in CI); running it end to end exposed and fixed a refusal-detection bug, which does not change any reported result, and two chunking problems for uploaded documents. Finally, we built a **NotebookLM-style web application** on the service (web, YouTube and text sources, source guides, a Studio with audio overviews, mind maps, flashcards and quizzes, sharing) with a choice of ten local open-weight models, whose refusal thresholds we calibrated per model: Qwen3-1.7B and Gemma-3-4B reached the best development F1 (.67 vs .62 for the research model). All code, predictions and scripts are available in the project repository.

Contents

Abstract **i**

1 Introduction **1**

1.1 Background . . . 1
1.2 Motivation . . . 1
1.3 Research questions . . . 1
1.4 Objectives . . . 1
1.5 Summary of outcomes . . . 2
1.6 Organisation of this report . . . 2

2 Literature Review **3**

2.1 Multi-turn RAG benchmarks . . . 3
2.2 Unanswerable questions and abstention . . . 3
2.3 Query rewriting and retrieval . . . 3
2.4 Generating unanswerable training data . . . 3
2.5 Gap addressed by this project . . . 3

3 Datasets **4**

3.1 Task definition . . . 4
3.2 MTRAG (training and development) . . . 4
3.3 MTRAG-UN (test) . . . 4
3.4 Document collections . . . 4
3.5 Self-created data: synthetic unanswerable examples . . . 5
3.6 The turn-position confound . . . 5

4 Models Used **6**

4.1 Large language model: Qwen2.5-3B-Instruct . . . 6
4.2 Retrieval models . . . 7
4.3 Cross-encoder: BGE-reranker-base . . . 7
4.4 DeBERTa-v3-base (earlier checker) . . . 7

5 System Design and Methodology **8**

5.1 Architecture . . . 8
5.2 Stage 1: query rewriting . . . 8
5.3 Stage 2: hybrid retrieval and re-ranking . . . 8
5.4 Stage 3: answer generation with prompt-only abstention . . . 8

5.5 Stage 4: answerability gates . . . 9
5.6 Evidence removal: generating unanswerable examples . . . 9
5.7 Systems compared . . . 10

6 Implementation **11**
6.1 Hardware and software environment . . . 11
6.2 Code organisation . . . 11
6.3 Implementation steps . . . 12
6.4 Checker training details . . . 12
6.5 Efficiency measures . . . 13

7 Experiments **14**
7.1 List of experiments . . . 14
7.2 Evaluation metrics . . . 14
7.3 Measuring abstention drift . . . 15
7.4 Controlled regression . . . 15
7.5 Controlled probe . . . 15

8 Results **16**
8.1 Main results . . . 16
8.2 RQ1: abstention drift . . . 17
8.3 RQ2: answerability gates . . . 18
8.4 Checker development . . . 19
8.5 Quality of the synthetic data . . . 19
8.6 Over-refusal . . . 20
8.7 Results by domain . . . 20

9 Agentic Extension: LangGraph Self-Correcting Agent (S5) **21**
9.1 Motivation . . . 21
9.2 Design . . . 21
9.3 Implementation with LangChain and LangGraph . . . 22
9.4 Selective self-check and calibration . . . 22
9.5 Results . . . 23
9.6 Clarifying questions on underspecified turns . . . 23
9.7 Example agent traces . . . 24
9.8 Summary . . . 24

10 Production Service: Deploying the S5 Agent **25**
10.1 Motivation and scope . . . 25
10.2 Architecture . . . 25

10.3 Uploads: section-aware chunking . . . 26
10.4 Guardrails . . . 26
10.5 LLMOps and DevOps . . . 27
10.6 Problems found by end-to-end testing . . . 27
10.7 Observed behaviour on the laptop . . . 28
10.8 Summary . . . 28

11 Notebook Application: A NotebookLM-Style Front End 29
11.1 Motivation and scope . . . 29
11.2 Features . . . 30
11.3 Architecture . . . 30
11.4 Sources beyond files . . . 30
11.5 Source guides and notebook-wide questions . . . 31
11.6 Studio outputs . . . 31
11.7 Choosing the model . . . 32
11.8 Refusal thresholds for every model . . . 33
11.9 End-to-end test on research papers . . . 34
11.10 Testing . . . 35
11.11 Summary . . . 36

12 Error Analysis 37

13 Challenges Faced and Solutions 39

14 Discussion 41
14.1 Why is there no abstention drift? . . . 41
14.2 Why did the answerability gates not help? . . . 41
14.3 Practical implications . . . 41

15 Limitations and Future Work 42
15.1 Limitations . . . 42
15.2 Future work . . . 42

16 Conclusion 43

Team Contributions 44

References 45

A Prompts 47

B Hyper-parameters 48

C Run Times **49**

D Commands to Reproduce the Results **50**

List of Figures

3.1 Share of unanswerable tasks by turn position in MTRAG and MTRAG-UN (our analysis). 5

5.1 Pipeline architecture. 8

8.1 Abstention recall on unanswerable turns by turn position, with 95% conversation-bootstrap confidence intervals. (a) MTRAG-UN test set; (b) controlled probe with question type held fixed. 17

8.2 Turns whose decision changes when each gate is added in front of S2. 18

8.3 False-abstention rate (answerable turns refused) by turn position, with 95% confidence intervals. 20

9.1 The S5 agent as a LangGraph state graph. 21

10.1 The production service around the S5 agent. 25

11.1 A notebook: sources with a source guide (left), a notebook-wide question answered from the source guides with one citation per source (middle), and the Studio with generated outputs (right). Screenshot from the development mode, in which a scripted stand-in replaces the LLM. 29

11.2 The notebook application on top of the service. 30

11.3 An interactive mind map generated from two sources (development mode). 32

11.4 The model menu: standard models with size, measured speed and the calibration shield (development mode, with a sample cloud catalogue). 33

List of Tables

3.1 MTRAG-UN test set used in this project, by domain and by turn bucket. 4
3.2 Document collections. 5
3.3 Answerability-checker data after filtering. 5

4.1 Models used in the project. The S5 agent orchestrates these models with LangChain and LangGraph (Chapter 9). 6

5.1 The six systems. Each adds one component to the previous one. 10

6.1 Hardware and software. 11
6.2 Main modules and scripts. 12

7.1 Experiments performed. 14

8.1 Main results. FA = share of answerable turns wrongly refused. Recall slope = change in abstention recall per extra turn. OR = odds ratio per turn from the controlled regression. Best system value per column in bold. 16
8.2 Abstention recall and false-abstention rate by turn bucket. 17
8.3 Checker development runs. “6 real” uses only the six real unanswerable development turns and is very noisy. 19
8.4 Synthetic examples judged still answerable by the Qwen2.5-3B LLM judge. 19
8.5 Abstention by domain on MTRAG-UN. 20

9.1 Choosing the self-check cut-off on the dev set (224 examples, 74 unanswerable). The dev rates are higher than on the test set because most dev unanswerables are synthetic. . 22
9.2 Agent results on MTRAG-UN (429 turns). The selective self-check cut-off (0.50) was chosen on dev. 23
9.3 What the agent changes compared with S2. 117 turns were retried. 23

10.1 Service components. 25
10.2 Guardrails. 26
10.3 Problems found by running the service end to end, and their fixes. 28

11.1 Features of the notebook application. 30
11.2 Studio outputs. 31
11.3 Per-model calibration on the MTRAG development examples. FA: answerable questions refused with the chosen thresholds; own refusals: refused by the model alone, with no gate and no self-check. Times include the answer and the self-check on the GTX 1650. . 33
11.4 End-to-end test on three research papers. Answerable: the general and the section question on each paper. Wrong refusals: which part of the agent refused an answerable question. Median s: time per chat question, including retrieval, gate, answer and self-check. 34

12.1 Qualitative error analysis on MTRAG-UN. 37

13.1 Main challenges during the project and how they were solved. 39

16.1 Division of work. 44

B.1 Hyper-parameters of the reported runs (`configs/laptop.yaml`). 48

C.1 Approximate run times. 49

1. Introduction

1.1 Background

Large language models (LLMs) can answer questions fluently, but they sometimes produce confident answers that are not supported by any source ("hallucinations"). *Retrieval-augmented generation* (RAG) reduces this problem: before answering, the system searches a document collection, retrieves the most relevant passages, and asks the LLM to answer using only those passages [4]. RAG is now the standard design for enterprise assistants, customer-support bots and document search tools.

RAG does not solve the problem completely. If the retrieved passages do not contain the answer, the model should reply "I don't know". In practice it often answers anyway, using partially related text or its own background knowledge. Declining to answer is called *abstention*. Good abstention matters in the domains our data covers (finance, government services, technical documentation), where a wrong answer can mislead a user.

1.2 Motivation

Two observations motivated this project.

1. **Abstention is weak even in strong systems.** In the SemEval-2026 Task 8 (MTRAGEval) shared task on multi-turn RAG, one of the best systems reached 83.9% answerability accuracy overall but detected only **21.8%** of the unanswerable turns on the development set [3].
2. **Abstention has mostly been studied on single questions** [5, 6, 7, 8]. Real users hold conversations: later questions are short, depend on earlier turns ("what about the second one?"), and the conversation history keeps growing. It is plausible that the assistant becomes *worse* at knowing when not to answer as the conversation continues. We call this *abstention drift*. No earlier work had measured it.

A second practical problem is data: unanswerable turns are rare (6.5% of the MTRAG training data), so training a detector needs a cheap way to create more examples.

1.3 Research questions

RQ1 – Abstention drift
Does detection of unanswerable questions get worse at later turns of a conversation?

RQ2 – A cheap fix
Can a lightweight answerability checker, trained on unanswerable examples generated cheaply by *evidence removal*, improve abstention without refusing more answerable questions (at most 5% of answerable development turns refused)?

1.4 Objectives

1. Build a reproducible multi-turn RAG pipeline with query rewriting, hybrid retrieval, re-ranking and grounded answer generation that runs on a laptop GPU.
2. Measure abstention per turn position with proper statistics: confidence intervals, a controlled regression, and a controlled probe that removes the confound of question type.

3. Design and train an answerability checker using real and synthetic unanswerable examples, and compare it with simpler alternatives.
4. Analyse errors and report what limits abstention in practice.

1.5 Summary of outcomes

- A working pipeline with six systems (S0–S4 and S2-rr) and a Gradio demo.
- RQ1: no abstention drift on the test set or under the controlled probe.
- RQ2: a negative result; synthetic data needed filtering (23% noise), and no checker improved the trade-off.
- A new finding: over-refusal of answerable questions is the main error (18–28%).
- An agentic extension (S5, LangGraph): retry before refusing lowers wrong refusals to 13.6%; with a calibrated self-check it reaches the best F1 (.511); it asks clarifying questions on 37% of underspecified turns.
- A production service around the S5 agent: streamed answers, persistent conversations, document uploads, guardrails (0 of 1,349 real questions blocked, injection attempts blocked), tracing, metrics and a CI quality gate, tested end to end on the laptop.
- A NotebookLM-style web application (sources, source guides, Studio, audio overviews, sharing) with a model menu of ten local models and refusal thresholds calibrated for each model.

1.6 Organisation of this report

Chapter 2 reviews related work. Chapter 3 describes the datasets and Chapter 4 the models used. Chapter 5 presents the system design and Chapter 6 the implementation. Chapter 7 describes the experiments and Chapter 8 the results. Chapter 9 presents the agentic extension built with LangChain and LangGraph, Chapter 10 the production service built around it, and Chapter 11 the NotebookLM-style application built on the service, including per-model refusal thresholds. Chapter 12 analyses errors, Chapter 13 lists the challenges we met and how we solved them, and Chapter 14 discusses the findings. Chapter 15 covers limitations and future work, and Chapter 16 concludes. The appendices list prompts, hyper-parameters and commands.

2. Literature Review

2.1 Multi-turn RAG benchmarks

Conversational question-answering datasets such as CoQA [13] and QuAC [14] include unanswerable questions, but each conversation is about one given passage, so there is no retrieval step. **MTRAG** [1] (TACL 2025) was the first benchmark of human-written multi-turn conversations over large document collections with answerability labels. **MTRAG-UN** [2] extends it with more unanswerable, underspecified and non-standalone questions and was used as the evaluation data for SemEval-2026 Task 8. Both benchmarks report performance on unanswerable questions as one overall number, and the SemEval system papers [3] focus on leaderboard scores; none analyses how abstention changes with turn position.

2.2 Unanswerable questions and abstention

SQuAD 2.0 [5] added unanswerable questions to extractive reading comprehension. *Selective prediction* lets a model decline when it is not confident [10, 11]. For LLMs, researchers have studied whether models know what they know [9], how several LLMs can collaborate to decide when to abstain [8], and surveyed abstention methods [7]. In RAG, the RGB benchmark tests "negative rejection" when documents lack the answer [6], and Self-RAG trains the model to critique its retrieved passages [12]. All of these evaluate isolated questions.

2.3 Query rewriting and retrieval

Follow-up questions are hard to search with directly. Rewriting them into standalone questions is a standard step in conversational search [15]. For retrieval we combine the classic BM25 ranking function [16] with dense embeddings from the BGE model family [17], merge the two rankings with reciprocal rank fusion [18], and re-rank the result with a cross-encoder [19]. Dense search uses the FAISS library [20].

2.4 Generating unanswerable training data

Unanswerable training examples are usually created by pairing questions with unrelated passages or by editing questions [5]. Our *evidence removal* keeps the real question and a realistic retrieval, and only removes the documents that contain the answer. To detect examples that remain answerable we use an LLM as a judge [23].

2.5 Gap addressed by this project

To our knowledge, no earlier work (i) measures abstention as a function of turn position while controlling for question type, or (ii) creates unanswerable training data for conversational RAG by removing evidence documents and measures how noisy that data is.

3. Datasets

3.1 Task definition

Each *task* is one user turn in a conversation, together with the earlier turns (the history). The system must either answer from the retrieved passages or abstain with a fixed "I don't know" reply. Every task carries a human label: ANSWERABLE, PARTIAL, UNANSWERABLE, CONVERSATIONAL or UNDERSPECIFIED. For our analysis:

- UNANSWERABLE $\rightarrow$ the system *should abstain* ($y = 1$);
- ANSWERABLE and PARTIAL $\rightarrow$ the system *should answer* ($y = 0$);
- CONVERSATIONAL and UNDERSPECIFIED turns are excluded.

3.2 MTRAG (training and development)

MTRAG [1] (TACL 2025, IBM Research, Apache-2.0 licence) contains 110 human-written conversations with 7.7 turns on average, converted into 842 tasks: 709 answerable, 68 partial, 55 unanswerable and 10 conversational. Each task includes the reference (gold) passages, a reference answer and a human-written standalone rewrite of the question. We used MTRAG to train and tune the answerability checker, split 80/20 *by conversation* (random seed 13) so that no conversation appears in both parts.

3.3 MTRAG-UN (test)

MTRAG-UN [2] (SemEval-2026 Task 8) contains 507 public tasks: 285 answerable, 47 partial, 97 unanswerable and 78 underspecified. It uses the same document collections. We used it only as the held-out test set. After removing underspecified turns, **429 tasks** remain: 97 unanswerable and 332 answerable (Table 3.1).

Domain	Unans.	Ans.	Total	Turn bucket	Unans.	Ans.
ClapNQ	22	83	105	Turns 1–2	13	90
Cloud	36	86	122	Turns 3–5	28	135
FiQA	12	58	70	Turns 6+	56	107
Govt	27	105	132			
All	97	332	429	All	97	332

Table 3.1: MTRAG-UN test set used in this project, by domain and by turn bucket.

3.4 Document collections

The four passage-level collections shipped with MTRAG contain about 366K passages in total. Each passage is a chunk of up to 512 tokens (median 1,465 characters).

Collection	Source	Character
ClapNQ	Wikipedia	General knowledge; the same topic often appears on many pages
Cloud	IBM Cloud documentation	Technical, specific pages
FiQA	Finance question-answer forums	Opinions and advice; many overlapping posts
Govt	U.S. government web pages	Services, regulations, public information

Table 3.2: Document collections.

3.5 Self-created data: synthetic unanswerable examples

From MTRAG we generated 507 synthetic unanswerable examples by evidence removal (Section 5.6) and kept 389 after filtering. Together with the real examples, the final checker data has 997 training and 224 development examples (Table 3.3).

Split	Answerable (real)	Unanswerable (real)	Unanswerable (synthetic)	Total
Train	627	49	321	997
Dev	150	6	68	224

Table 3.3: Answerability-checker data after filtering.

3.6 The turn-position confound

Unanswerable questions are not spread evenly over turns (Figure 3.1). In MTRAG-UN, 33.7% of turns 6 and later are unanswerable, compared with 9.4% of turns 1–2, because annotators usually asked unanswerable questions after a topic had been established. If we simply compare recall across turn positions, we mix the effect of conversation depth with the effect of *which* questions appear late. Our evaluation controls for this (Sections 7.4 and 7.5).

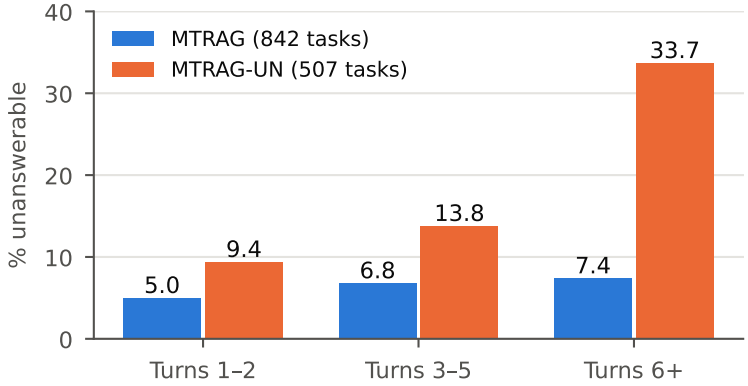


Figure 3.1: Share of unanswerable tasks by turn position in MTRAG and MTRAG-UN (our analysis).

4. Models Used

This chapter lists every model used in the project, its role and how it was run. Table 4.1 is the summary.

Model	Type	Size	Role in the project
Qwen2.5-3B-Instruct [21] (`qwen2.5:3b`, Q4_K_M)	Large language model (decoder)	3.1B params	(1) query rewriting, (2) answer generation with prompt-only abstention, (3) LLM judge for filtering synthetic data; (4) in the S5 agent: retry rewriting, answer self-check and clarifying questions. Used in all systems.
BM25 (`bm25s`) [16]	Sparse lexical ranking	–	First-stage retrieval in all systems.
BAAI/bge-small-en-v1.5 [17]	Dense bi-encoder (embeddings)	33M params, 384-dim	Dense retrieval in S2–S4.
BAAI/bge-reranker-base [17]	Cross-encoder (XLM-RoBERTa-base)	278M params	(1) re-ranking in S2–S4; (2) untrained gate in S2-rr; (3) initial weights of the final fine-tuned checkers in S3 and S4.
microsoft/deberta-v3-base [22]	Encoder (cross-encoder classifier)	184M params	Earlier answerability checkers (5 training runs); replaced by the re-ranker-based checker.

Table 4.1: Models used in the project. The S5 agent orchestrates these models with LangChain and LangGraph (Chapter 9).

4.1 Large language model: Qwen2.5-3B-Instruct

What it is. Qwen2.5-3B-Instruct [21] is an open instruction-tuned LLM from Alibaba's Qwen team with about 3.1 billion parameters and a 32K-token context window. It follows chat instructions well for its size and is released under a permissive licence.

Why we chose it. Our GPU (NVIDIA GTX 1650) has only 4 GB of memory. A 7B model does not fit in 4-bit precision next to the retrieval models, while a 3B model quantised to 4 bits needs about 1.9 GB. Our configuration also supports Qwen2.5-7B-Instruct for a larger GPU, but that setting was not run.

How it was run. We first loaded it with Hugging Face Transformers and bitsandbytes 4-bit quantisation, which took about 161 seconds per task on the laptop. We then switched to the **Ollama** runtime (llama.cpp) with the GGUF `Q4_K_M` quantisation of the same model (`ollama pull qwen2.5:3b`), which is several times faster. Decoding is greedy (temperature 0, fixed seed 13), so outputs are deterministic. The context window was set to 4,096 tokens.

Its three roles.

1. **Query rewriter:** turns a follow-up question into a standalone search query (max 64 new tokens).
2. **Answer generator:** writes the answer from the top-5 passages, or replies "I'm sorry, but I don't have the answer to your question." when the passages are insufficient (max 128 new tokens). This is the *prompt-only abstention* used by every system.
3. **LLM judge:** decides whether a synthetic "unanswerable" example is in fact still answerable (max 3 new tokens: "yes"/"no").
4. **Agent reasoning (S5):** writes alternative queries for retries, checks whether an answer is supported

by the passages, and writes clarifying questions (Chapter 9).

The prompts are listed in Appendix A.

4.2 Retrieval models

BM25 scores passages by overlapping words, weighted by how rare each word is. It needs no training and is strong on exact terms such as product names. We used the `bm25s` library with English stop words and took the top 100 passages.

BGE-small-en-v1.5 [17] is a 33M-parameter embedding model that maps queries and passages to 384-dimensional vectors; similar meanings have similar vectors, so it finds paraphrases that BM25 misses. All 366K passages were encoded once (in fp16 on the GPU) and stored in FAISS flat inner-product indexes [20]. Queries are prefixed with the instruction recommended by the model authors.

4.3 Cross-encoder: BGE-reranker-base

A *cross-encoder* reads the query and a passage together and outputs one relevance score. This is more accurate than comparing two separate embeddings but slower, so it is applied only to the top 15 fused candidates. `bge-reranker-base` [17] is based on XLM-RoBERTa-base (278M parameters). We ran it in fp16 with a maximum of 256 tokens per pair. The same model plays three roles: re-ranker, training-free abstention gate (S2-rr), and starting point of the fine-tuned checker (S3, S4).

4.4 DeBERTa-v3-base (earlier checker)

DeBERTa-v3-base [22] is a 184M-parameter encoder that performs well on classification. Our first checker design fine-tuned it to classify (question, passages) pairs as answerable or unanswerable. It never matched the untrained re-ranker (Section 8.4), so it was replaced.

5. System Design and Methodology

5.1 Architecture

Figure 5.1 shows the pipeline. A question can be refused in two places: at the *answerability gate* before generation, or by the *generator* itself.

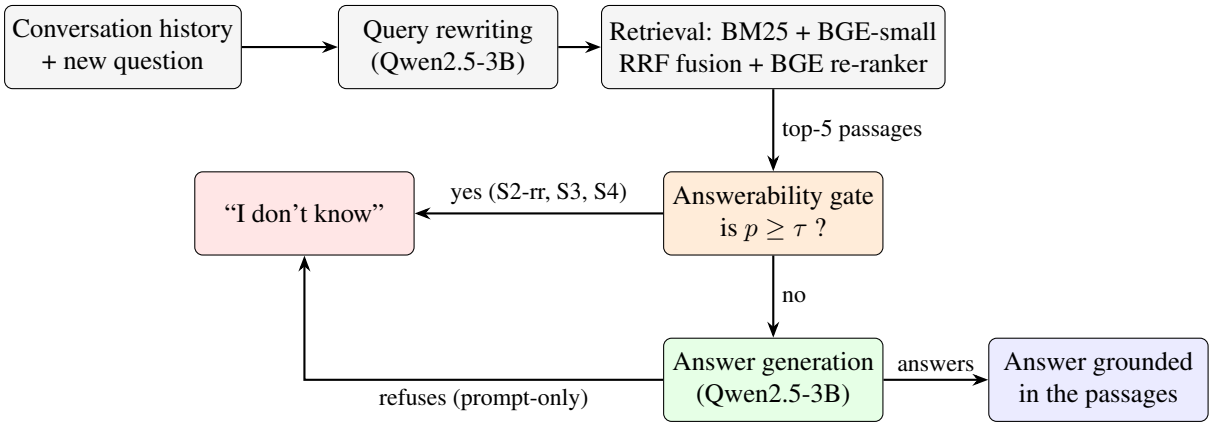


Figure 5.1: Pipeline architecture.

5.2 Stage 1: query rewriting

The rewriter gives Qwen2.5-3B the last three exchanges of the history (each message cut to 400 characters) and the latest question, and asks for one standalone question. The first turn of a conversation is used unchanged. System S0 skips this step and searches with the raw last user turn.

5.3 Stage 2: hybrid retrieval and re-ranking

For each query, BM25 and the dense retriever each return 100 passages. Their rankings are merged by reciprocal rank fusion [18]:

$$\mathrm{RRF}(d) = \sum_{r \in \{\text{BM25, dense}\}} \frac{1}{60 + \mathrm{rank}_r(d)}.$$

The top 15 fused passages are re-scored by the BGE re-ranker, and the best $K = 5$ are kept with their scores $s_1, \ldots, s_5 \in [0, 1]$.

5.4 Stage 3: answer generation with prompt-only abstention

The generator sees the five passages (each cut to 1,000 characters), the last three exchanges and the question. It is told to answer only from the passages and otherwise to give the fixed refusal. A reply counts as a refusal if its first 200 characters match one of six refusal patterns (for example "I don't have the answer" or "the passages do not contain").

5.5 Stage 4: answerability gates

A gate computes $p = P(\text{unanswerable})$ from the rewritten query and the five passages; if $p \geq \tau$ the system abstains without calling the generator.

Untrained re-ranker gate (S2-rr). $p = 1 - \max_i s_i$: abstain when even the best passage is barely relevant.

Fine-tuned checker (S3, S4). The checker scores each (query, passage) pair separately and takes the minimum:

$$p = \min_{i=1,\ldots,5} P_\theta(\text{passage } i \text{ does not answer the query}), \qquad P_\theta = 1 - \sigma\big(z_\theta(q, P_i)\big),$$

where z_θ is the relevance logit of the fine-tuned BGE re-ranker. A turn is judged answerable if *any* passage answers it. Training pairs: every passage of an unanswerable example is a "does not answer" pair; the gold passages of an answerable example are "answers" pairs. S3 uses real labels only; S4 adds the filtered synthetic examples.

Threshold. On the development set we choose

$$\tau^* = \arg\max_\tau \mathrm{F1}_{\text{abstain}}(\tau) \quad \text{subject to} \quad \mathrm{FA}(\tau) \leq 5\%,$$

where FA is the share of answerable development turns that the gate refuses. The chosen thresholds were 0.895 (S2-rr), 0.543 (S3) and 0.611 (S4).

5.6 Evidence removal: generating unanswerable examples

For each answerable MTRAG turn we know which passages contain the answer. Algorithm 1 removes *all* passages from those source documents, retrieves again, and labels the new context unanswerable. This costs no annotation and produces realistic, closely related contexts. Answerable training examples are built the same way (retrieval with the gold passages inserted at random positions), so the checker cannot learn to spot gold passages by their position or style.

Algorithm 1 Evidence removal for one answerable turn

Require: turn x with rewritten query q, gold passages G, reference answer a

$\mathcal{D} \leftarrow$ source documents of the passages in G
$R \leftarrow \textsc{Retrieve}(q)$, excluding every passage of the documents in $\mathcal{D}$
$C \leftarrow$ top-5 passages of R
if more than half of the content words of a appear in C **then return** none ▷ lexical filter
end if
if $\textsc{LLMJudge}(x, a, C)$ = "yes" **then return** none ▷ still answerable
end if
return $(q, C, \textsc{unanswerable})$

5.7 Systems compared

System	Components	Models involved
S0	Last user turn as query; BM25; prompt-only refusal	BM25, Qwen2.5-3B
S1	S0 + LLM query rewriting	BM25, Qwen2.5-3B
S2	S1 + dense retrieval, RRF fusion, re-ranking	+ BGE-small, BGE-reranker
S2-rr	S2 + gate on the untrained re-ranker score	same as S2
S3	S2 + fine-tuned checker, real labels only	+ fine-tuned BGE-reranker
S4	S2 + fine-tuned checker, real + synthetic labels (proposed)	+ fine-tuned BGE-reranker

Table 5.1: The six systems. Each adds one component to the previous one.

Because generation is deterministic and the gate sits before the generator, S2-rr, S3 and S4 give exactly the same outputs as S2 except where their gate fires. We therefore ran S2 once and derived the gated systems from its stored queries and passages.

6. Implementation

6.1 Hardware and software environment

Item	Details
Machine	Laptop, NVIDIA GeForce GTX 1650 (4 GB VRAM, no bf16 support), 16 GB RAM, 512 GB SSD (OS) + 1 TB HDD (project, data, models)
Operating system	Windows 11, Anaconda PowerShell Prompt, conda environment `abstain`
Language	Python 3.11
Deep learning	PyTorch 2.14 with CUDA 12.6; Hugging Face Transformers 5; Accelerate; Datasets
Retrieval	`bm25s`, `sentence-transformers`, FAISS (CPU)
LLM serving	Ollama 0.34 (llama.cpp) for Qwen2.5-3B-Instruct GGUF; bitsandbytes for the Transformers backend
Agent framework	LangGraph 1.2, langchain-core 1.6, langchain-ollama 1.1 (`ChatOllama`)
Analysis	NumPy, pandas, scikit-learn (logistic regression), Matplotlib
Demo and tests	Gradio (interactive demo), pytest (26 research and 23 service tests), Ruff
Service (Ch. 10)	FastAPI, SQLAlchemy 2 + Alembic, PostgreSQL, Qdrant 1.19, Celery + Redis, Guardrails AI 0.11, Langfuse 3, OpenTelemetry, Prometheus, Grafana, Docker Compose, uv, GitHub Actions

Table 6.1: Hardware and software.

6.2 Code organisation

The code is a Python package with one script per step. Settings live in YAML configuration files: `configs/default.yaml` for a large GPU and `configs/laptop.yaml` for the 4 GB laptop. In total the research package and scripts are about 2,450 lines of Python; the production service (Chapter 10) adds about 3,200.

Module / script	Purpose
`src/abstain/data.py`	Load MTRAG / MTRAG-UN tasks and corpora, conversation split, turn buckets
`src/abstain/retrieval.py`	BM25 and dense indexes, RRF fusion, re-ranker, hybrid retriever
`src/abstain/llm.py`	LLM backends: Transformers (4-bit) and Ollama
`src/abstain/rewrite.py`	Query rewriting with the LLM
`src/abstain/generate.py`	Grounded answer generation prompt
`src/abstain/idk.py`	Refusal response and refusal detector
`src/abstain/checker.py`	Checker training and inference, re-ranker gate, threshold selection
`src/abstain/synth.py`	Evidence removal, lexical support filter, LLM judge
`src/abstain/pipeline.py`	End-to-end system with per-stage timing
`src/abstain/metrics.py`	Abstention metrics, drift slope with bootstrap, controlled regression
`scripts/*.py`	Build indexes, make and filter checker data, train checkers, run systems, apply gates, run probe, evaluate
`src/abstain/agent.py`	S5 LangGraph agent: LangChain wrappers, state graph, self-check, clarification
`app/app.py`	Gradio demo showing sources and a $P(\text{unanswerable})$ meter
`service/src/abstain_api/`	Production service: API routers, agent runtime, retrieval (Qdrant/pgvector), uploads, guardrails, tracing, metrics, evaluation
`service/eval/`, `infra/`, `deploy/`	Quality-gate sample and baseline; Prometheus/Grafana configuration; Fly.io, Render, Kubernetes, Modal and RunPod deployment files

Table 6.2: Main modules and scripts.

6.3 Implementation steps

1. **Data setup:** download the MTRAG benchmark repository, compute dataset statistics.
2. **Indexing:** build BM25 and dense (BGE-small + FAISS) indexes for all four collections.
3. **Checker data:** rewrite queries with Qwen, retrieve, create answerable and evidence-removed examples.
4. **Filtering:** apply the LLM judge to every synthetic example (with a resumable cache).
5. **Checker training:** fine-tune the BGE re-ranker (S4: real + synthetic; S3: real only); fit the S2-rr threshold.
6. **System runs:** run S0, S1 and S2 on MTRAG-UN; derive S2-rr, S3 and S4 from S2.
7. **Probe and evaluation:** run the controlled probe and compute all metrics, confidence intervals and plots.

6.4 Checker training details

- Initial weights: `BAAI/bge-reranker-base` (one relevance logit).
- Loss: binary cross-entropy with class weights (the minority class is up-weighted by the class-frequency ratio).
- Optimiser: AdamW, learning rate 10^{-5}, 10% warm-up, weight decay 0.01, 2 epochs, batch size 8 with 2 gradient-accumulation steps (effective 16), fp16, 256 tokens per pair.

- Training pairs: 3,525 for S4 (1,849 “does not answer”), 1,921 for S3 (245 “does not answer”); 779 development pairs.
- The checkpoint with the lowest development loss was kept. Training took 74 minutes (S4) and 41 minutes (S3).

6.5 Efficiency measures

Running an LLM pipeline on a 4 GB GPU required several optimisations (details in Chapter 13):

- Ollama instead of bitsandbytes for the LLM: from about 161 s to about 35 s per task.
- Shorter prompts (three history exchanges, 1,000-character passages): from 35.4 s to 24.2 s per task.
- Deriving the gated systems from the S2 run: about 10 minutes per gate instead of about 3 hours per system.
- Unloading the Ollama model from GPU memory before checker training.

7. Experiments

7.1 List of experiments

#	Experiment	Purpose	Data
1–6	Systems S0, S1, S2, S2-rr, S3, S4	Abstention quality and drift (RQ1, RQ2)	MTRAG-UN, 429 turns
7	Controlled probe: generator	Drift with question type fixed (RQ1)	MTRAG train+dev, 1,221 examples
8	Controlled probe: checker	Drift of the S4 gate (RQ1)	MTRAG dev, 224 examples
9–13	DeBERTa-v3 checkers (5 runs)	Checker development	MTRAG dev (256)
14–16	BGE re-ranker: untrained, fine-tuned real only, fine-tuned real+synthetic	Checker development (RQ2)	MTRAG dev (224)
17	LLM-judge filtering of synthetic data	Label-noise measurement	507 synthetic examples
18	Refusal-detector sensitivity analysis	Robustness of RQ1 conclusions	MTRAG-UN
19	Dataset statistics by turn	Confound analysis	MTRAG, MTRAG-UN
20–23	S5 agent: retry only, retry + self-check (always), self-check without retry, retry + selective self-check	Agentic extension	MTRAG-UN, 429 turns
24	Self-check cut-off calibration	Choose min_p on dev	MTRAG dev, 224 examples
25	Clarification on underspecified turns	Agentic extension	MTRAG-UN, 78 turns

Table 7.1: Experiments performed.

7.2 Evaluation metrics

With $y_i = 1$ for unanswerable turns and $\hat{a}_i = 1$ when the system abstains (at the gate or in the generator):

- **Abstention recall** $= P(\hat{a} = 1 \mid y = 1)$: share of unanswerable turns correctly refused.
- **Abstention precision** $= P(y = 1 \mid \hat{a} = 1)$: share of refusals that were justified.
- **F1**: harmonic mean of abstention precision and recall.
- **False-abstention rate (FA)** $= P(\hat{a} = 1 \mid y = 0)$: share of answerable turns wrongly refused (lower is better).
- **AUROC** (for checkers): probability that a random unanswerable example gets a higher p than a random answerable one (0.5 = chance, 1.0 = perfect).

All metrics are also reported per turn bucket: early (turns 1–2), mid (3–5), late (6+).

7.3 Measuring abstention drift

The **drift slope** β is estimated from unanswerable turns with a linear-probability model

$$P(\hat{a}_i = 1) = \alpha + \beta\, t_i,$$

where t_i is the turn number. A negative β means recall falls by $|\beta|$ per extra turn. Turns of one conversation are correlated, so the 95% confidence interval comes from 1,000 bootstrap resamples of *whole conversations*.

7.4 Controlled regression

A logistic regression (scikit-learn) of $\hat{a}_i$ on t_i with the domain (4 levels) and follow-up type (follow-up, clarification, none) as additional inputs separates the turn effect from question type. We report the **odds ratio per turn**: below 1 means abstention becomes less likely at later turns.

7.5 Controlled probe

The probe holds question type fixed. Every answerable MTRAG turn gives two versions of the same question at the same turn position: one with the gold passages inserted (answerable) and one with the evidence removed (unanswerable). Only the passages differ. We fed these fixed inputs (a) to the **generator**, with the real conversation history, on train and development conversations (1,221 examples from 108 conversations; the generator is never trained, so using the training split is valid), and (b) to the **S4 checker**, on the development split only (224 examples from 22 conversations), because the checker was trained on the training split.

8. Results

8.1 Main results

	Abstention (MTRAG-UN, $n = 429$)				Drift per turn (95% CI)		Controlled
System	F1	Recall	Prec.	FA ↓	Recall slope	FA slope	OR/turn
S0	.451	**.567**	.374	.277	−.002 [−.048, +.046]	−.016 [−.037, +.004]	0.99
S1	.453	.526	.398	.232	−.005 [−.051, +.041]	−.020 [−.040, −.000]	0.97
S2	.443	.464	**.425**	**.184**	−.003 [−.046, +.043]	−.006 [−.022, +.012]	1.00
S2-rr	**.473**	.546	.417	.223	+.011 [−.031, +.057]	−.000 [−.018, +.018]	1.06
S3	.457	.515	.410	.217	+.011 [−.030, +.055]	−.002 [−.019, +.018]	1.06
S4	.449	.495	.410	.208	+.008 [−.034, +.051]	+.002 [−.015, +.020]	1.04
Probe: generator	.489	.423	.578	.176	+.011 [−.010, +.029]	+.006 [−.005, +.016]	1.08
Probe: checker	.380	.257	.731	.047	+.000 [−.040, +.054]	+.006 [−.008, +.022]	1.01

Table 8.1: Main results. FA = share of answerable turns wrongly refused. Recall slope = change in abstention recall per extra turn. OR = odds ratio per turn from the controlled regression. Best system value per column in bold.

Table 8.1 gives the main results. Abstention is hard for every system: F1 ranges from .443 to .473. The systems move along a trade-off rather than improving on it:

- S0 (weakest retrieval) refuses most often: highest recall (.567) but also the highest false-abstention rate (27.7%).
- Better retrieval (S1, S2) lowers false abstention to 18.4% but also lowers recall to .464, because better passages also give the generator more material to answer unanswerable questions from.
- The gates (S2-rr, S3, S4) move S2 back towards more refusals.

8.2 RQ1: abstention drift

	Turns 1–2		Turns 3–5		Turns 6+	
System	Recall	FA	Recall	FA	Recall	FA
S0	.538	.289	.536	.319	.589	.215
S1	.462	.267	.500	.252	.554	.178
S2	.538	.156	.393	.222	.482	.159
S2-rr	.538	.178	.464	.244	.589	.234
S3	.538	.178	.429	.259	.554	.196
S4	.538	.167	.393	.237	.536	.206
n (unans. / ans.)	13 / 90		28 / 135		56 / 107	
Probe: generator	.404	.154	.401	.178	.460	.192
n	104 / 208		177 / 298		163 / 271	
Probe: checker	.312	.024	.182	.050	.320	.062
n	16 / 42		33 / 60		25 / 48	

Table 8.2: Abstention recall and false-abstention rate by turn bucket.

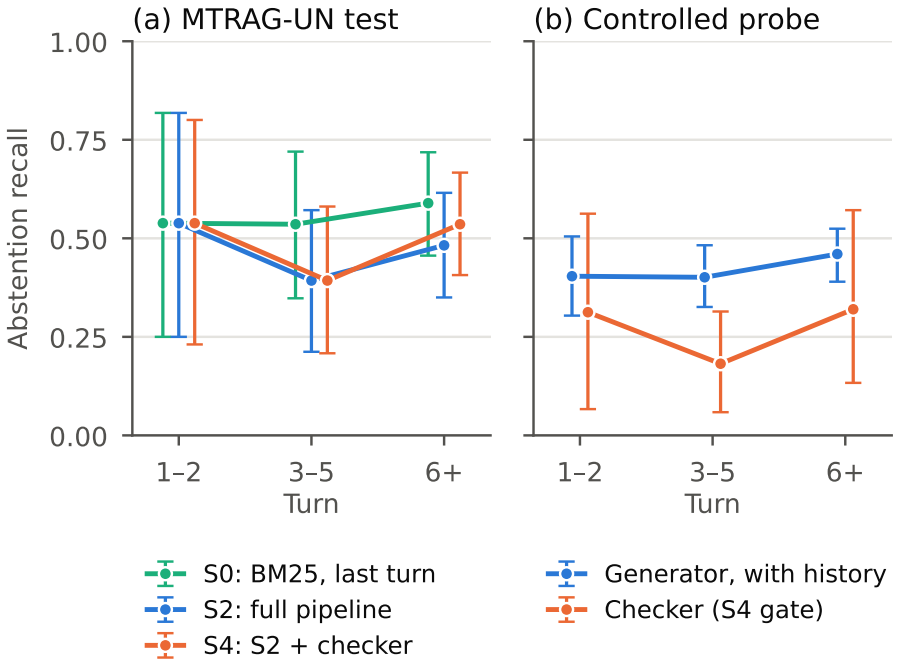


Figure 8.1: Abstention recall on unanswerable turns by turn position, with 95% conversation-bootstrap confidence intervals. (a) MTRAG-UN test set; (b) controlled probe with question type held fixed.

Finding: there is no abstention drift.

- **Test set.** All six recall slopes lie between -0.005 and $+0.011$ per turn, and every 95% confidence interval contains zero. With domain and follow-up type controlled, the odds ratios are 0.97–1.06, all close to 1. The per-bucket values (Table 8.2, Figure 8.1a) show no downward trend. Because the early bucket has only 13 unanswerable turns, the test set alone can only exclude large drifts (about ± 0.045 per turn).

- **Controlled probe.** With 444 unanswerable examples spread evenly across turns and question type fixed, the generator's recall is .404, .401 and .460. The slope is $+0.011$ $[-0.010, +0.029]$ and the odds ratio 1.08. This rules out declines larger than about one percentage point per turn; if anything, the generator refuses slightly *more* later in a conversation.
- **Checker.** The checker never sees the conversation history, so it cannot drift by design. Its probe curve is flat (slope $+0.000$).

Robustness check. The refusal detector misses some hedged refusals ("I do not have specific information about …"): 20 of the 429 S2 replies. Counting them as refusals raises S2's recall to .577 and FA to .211 (F1 .502), and S4's to .598 and .232 (F1 .500), but the drift slopes stay near zero: $+0.016$ $[-0.031, +0.065]$ for S0, $+0.002$ $[-0.041, +0.050]$ for S2 and $+0.010$ $[-0.033, +0.058]$ for S4.

8.3 RQ2: answerability gates

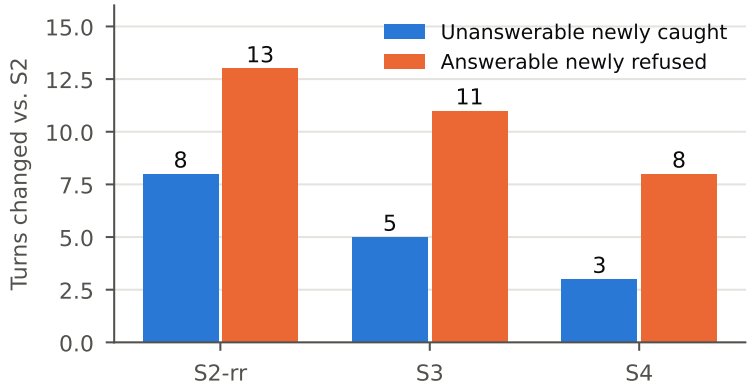


Figure 8.2: Turns whose decision changes when each gate is added in front of S2.

Finding: the gates do not pay off. A gate can only add refusals to S2. Compared with S2 (Figure 8.2):

- S2-rr (untrained re-ranker) fires on 51 turns; it catches 8 more unanswerable turns but wrongly refuses 13 more answerable ones.
- S3 (real labels only) catches 5 more and wrongly refuses 11 more.
- S4 (real + synthetic labels) catches 3 more and wrongly refuses 8 more.

Every gate adds more wrong refusals than correct ones. F1 still rises slightly (to .473, .457 and .449) because recall on the rare unanswerable class carries much weight, but users would see more unjustified "I don't know" replies.

Budget transfer. Thresholds were chosen to refuse at most 5% of answerable *development* turns, yet test FA rose by 3.9 (S2-rr), 3.3 (S3) and 2.4 (S4) percentage points: the development data (MTRAG, partly synthetic) differ from the test data (MTRAG-UN).

Real vs. synthetic labels. On the development set, fine-tuning on the 49 real unanswerable examples alone *lowered* the re-ranker's AUROC from .741 to .660 (over-fitting to very few examples), while adding the filtered synthetic examples avoided this (.747). On the test set, however, S4 did not beat the untrained S2-rr.

8.4 Checker development

#	Checker	Dev AUROC (all / 6 real unans.)	
		Unfiltered dev (256)	Filtered dev (224)
1	DeBERTa-v3, passages concatenated, real + synthetic	.658 / .563	–
2	DeBERTa-v3, passages concatenated, real only	.562 / .498	–
3	DeBERTa-v3, one passage at a time, real + synthetic	.651 / .526	–
4	DeBERTa-v3, one passage at a time, real only	.538 / .624	–
5	BGE re-ranker, untrained (S2-rr)	–	.741 / .619
6	BGE re-ranker, fine-tuned, real only (S3)	–	.660 / .623
7	BGE re-ranker, fine-tuned, real + synthetic (S4)	–	**.747 / .654**

Table 8.3: Checker development runs. "6 real" uses only the six real unanswerable development turns and is very noisy.

We tried three designs (Table 8.3):

1. **DeBERTa-v3, all passages concatenated.** The five passages were cut to 600 characters each and joined into one 384-token input. In 27 of 150 answerable development examples the gold passage was in position 3–5 and was partly or completely cut off. AUROC .66.
2. **DeBERTa-v3, one passage at a time.** Removed the truncation problem but did not help (.651).
3. **BGE re-ranker as the starting point.** The untrained re-ranker already reached .741; fine-tuning on real + synthetic data reached .747.

Starting from a relevance model mattered more than the input design or the training data. Tuning the threshold on the six real unanswerable development turns alone (our first approach) was unreliable, since one example changes recall by 17 points; we therefore tuned on the full development set.

8.5 Quality of the synthetic data

Collection	Generated	Still answerable (dropped)	Share
ClapNQ (Wikipedia)	180	49	27%
FiQA (finance)	142	33	23%
Govt (government)	117	26	22%
Cloud (IBM docs)	68	10	15%
All	507	118	23%

Table 8.4: Synthetic examples judged still answerable by the Qwen2.5-3B LLM judge.

Reading the synthetic examples showed many that could still be answered. For instance, "Who funds the Europa Clipper mission?" still retrieved several pages titled "NASA's Europa Clipper", and "What intern programs does NASA have?" retrieved three JPL "STEM Internships" pages. Removing the source *document* does not remove the *fact* when the collection repeats it on other pages. The LLM judge dropped 118 of 507 examples (23%, Table 8.4), most often in ClapNQ, where Wikipedia covers popular topics on many pages, and least in Cloud, whose documentation pages are more specific. After filtering, 321 training and 68 development synthetic examples remained (from 407 and 100).

The lexical filter alone was not enough. Reference answers of real answerable turns share most of their words with their passages (median overlap .87–.90) and synthetic examples far fewer (.30–.32), yet many low-overlap synthetic examples were still answerable through paraphrase.

8.6 Over-refusal

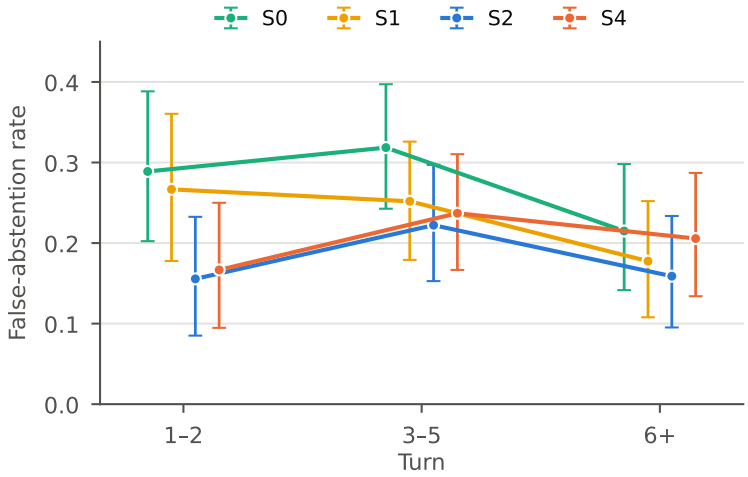


Figure 8.3: False-abstention rate (answerable turns refused) by turn position, with 95% confidence intervals.

Finding: over-refusal is the largest error. Even the best retrieval (S2) refuses 18.4% of answerable turns (61 of 332), while it misses 52 of the 97 unanswerable turns. Without query rewriting, wrong refusals are highest early and mid-conversation (S0: .289, .319, .215). With rewriting (S1) they fall steadily from .267 to .178 (slope -0.020 per turn, the only confidence interval that excludes zero), possibly because opening questions are short and generic and retrieve less specific passages (Figure 8.3).

8.7 Results by domain

	S2		S4	
Domain (unans. / ans.)	Recall	FA	Recall	FA
ClapNQ (22 / 83)	.41	.23	.41	.24
Cloud (36 / 86)	.44	.09	.50	.12
FiQA (12 / 58)	.58	.14	.58	.17
Govt (27 / 105)	.48	.25	.52	.28

Table 8.5: Abstention by domain on MTRAG-UN.

Over-refusal is concentrated in ClapNQ and Govt (23–28% FA), whose varied web and Wikipedia pages often contain the answer in a form the 3B generator does not recognise. Cloud documentation is refused least (9%). The S4 gate helps most in Cloud (+6 points recall for +3 points FA) and not at all in ClapNQ.

9. Agentic Extension: LangGraph Self-Correcting Agent (S5)

9.1 Motivation

The fixed pipeline (S2–S4) rewrites the question once, retrieves once, and then either answers or refuses. Chapter 8 (and the error analysis in Chapter 12) showed where this fails: the largest error is *over-refusal* (18.4% of answerable turns for S2), and several refusals start with a bad query rewrite. A human assistant who cannot find an answer would search again with different words, ask what the user means, or double-check a doubtful answer. We added these behaviours as a small *agent* built with LangChain's LangGraph framework.

9.2 Design

The agent is a LangGraph `StateGraph`: each step is a node, the state (question, history, queries tried, passages, gate score, answer, decision, trace) flows between nodes, and conditional edges decide what happens next (Figure 9.1).

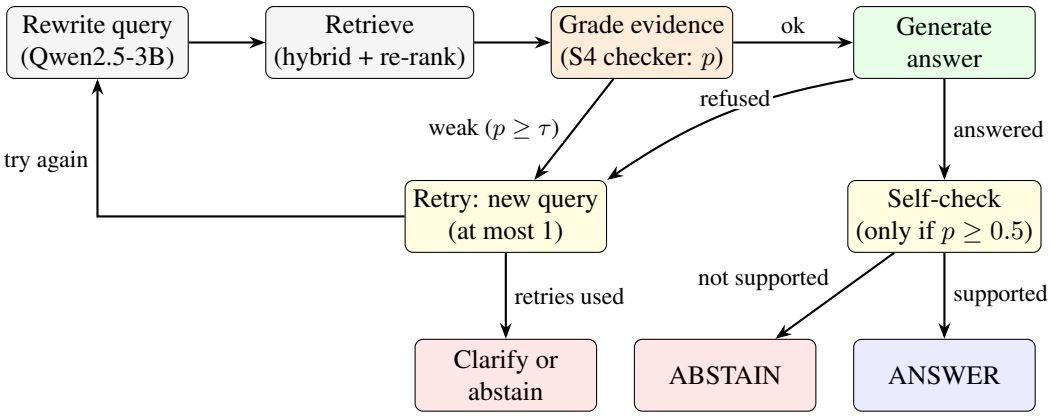


Figure 9.1: The S5 agent as a LangGraph state graph.

1. **First attempt.** Rewrite, retrieve and grade exactly as in S2/S4; the S4 checker gives the gate score p ($\tau = 0.611$).
2. **Retry before refusing.** If the evidence is weak ($p \geq \tau$) or the generator refuses, Qwen writes a *different* query, told to expand abbreviations, keep the user's own key words and not carry over an earlier topic, and is shown the queries already tried. The agent retrieves and grades again (one retry).
3. **Clarify or abstain.** If the retry also fails, Qwen judges whether the question is ambiguous. If yes, the agent asks one short clarifying question; otherwise it abstains. In the binary metrics a clarifying question counts as not answering.
4. **Selective self-check.** After an answer, Qwen judges whether the answer's main claim is supported by the passages; if not, the agent abstains. The check runs only when the gate already doubts the evidence ($p \geq$ min_p, Section 9.4).

We deliberately used **fixed, rule-based routing** rather than a free "tool-calling" (ReAct) agent: a 3B model follows explicit steps reliably but chooses tools poorly, and fixed routing keeps the agent easy to evaluate and explain.

9.3 Implementation with LangChain and LangGraph

- **LLM:** Qwen2.5-3B-Instruct through LangChain's `ChatOllama` (`langchain-ollama`), temperature 0, seed 13, with automatic retries on server errors. The existing rewriting and generation prompts are reused unchanged.
- **Retriever:** a LangChain `BaseRetriever` subclass wrapping our hybrid BM25 + BGE-small + re-ranker retriever, returning LangChain `Document` objects.
- **Graph:** a LangGraph `StateGraph` with seven nodes (`rewrite`, `retrieve`, `grade`, `generate`, `self_check`, `retry`, `clarify_or_abstain`) and three conditional edges. Code: `src/abstain/agent.py` (about 300 lines); versions LangGraph 1.2, langchain-core 1.6, langchain-ollama 1.1.
- **Trace:** every prediction stores the queries tried, the gate scores, each decision and the final outcome, so every agent decision can be inspected.
- **Efficient evaluation:** the agent's first attempt is identical to S2, so `run_agent.py` reuses the stored S2 queries, passages and answers and the S4 gate scores; only the new steps call the LLM. The run is resumable.
- **Tests:** seven unit tests drive the graph with a scripted fake LLM and check every route (answer, retry, clarify, abstain, self-check rejection, selective skip).

9.4 Selective self-check and calibration

Our first version self-checked *every* answer. The 3B judge said "not supported" for 66% of answers to answerable questions, barely more often for unanswerable ones (81%), and pushed false abstention to 70.8% (Table 9.2). We therefore run the self-check only when the gate score is already doubtful, $p \geq$ min_p.

The cut-off was chosen on the MTRAG development set, never on the test set, using the same 224 dev examples that set the gate threshold. For each example we stored the gate score, the generated answer and the self-check verdict (73 minutes), and evaluated every cut-off (Table 9.1). We picked the best dev F1 among cut-offs that add at most 5 percentage points of false abstention: **min_p = 0.50**. (Cut-offs 0.45 and 0.35 had slightly higher dev F1 but exceeded the false-abstention limit.)

Self-check used when	Recall	FA	Precision	F1
never	.662	.253	.563	.609
$p \geq 0.60$	.662	.260	.557	.605
$p \geq 0.55$	.689	.273	.554	.614
$p \geq 0.50$ **(chosen)**	**.730**	**.300**	**.545**	**.624**
$p \geq 0.45$	.770	.340	.528	.626
$p \geq 0.35$	.851	.393	.516	.643
$p \geq 0.25$	.865	.467	.478	.615
always ($p \geq 0$)	.959	.720	.397	.561

Table 9.1: Choosing the self-check cut-off on the dev set (224 examples, 74 unanswerable). The dev rates are higher than on the test set because most dev unanswerables are synthetic.

9.5 Results

System	F1	Recall	Precision	FA ↓	Recall slope / turn [95% CI]
S2 (no agent)	.443	.464	.425	.184	−.003 [−.046, +.043]
S2-rr (best gate, Ch. 8)	.473	.546	.417	.223	+.011 [−.031, +.057]
S4 (checker gate)	.449	.495	.410	.208	+.008 [−.034, +.051]
S5: retry only	.465	.443	**.489**	**.136**	+.010 [−.033, +.054]
S5: retry + selective self-check	**.511**	.577	.459	.199	+.020 [−.026, +.068]
S5: retry + self-check always	.415	.897	.270	.708	−.006 [−.036, +.026]
S5: self-check always, no retry	.415	.907	.269	.720	−.004 [−.034, +.028]

Table 9.2: Agent results on MTRAG-UN (429 turns). The selective self-check cut-off (0.50) was chosen on dev.

Change compared with S2	Retry only	Retry + selective self-check
Answerable turns recovered (S2 refused, agent answers)	16	16
Correct refusals lost (unanswerable, agent now answers)	2	2
Unanswerable turns newly caught	0	13
Answerable turns newly refused	0	21
Net correct decisions gained	+14	+6

Table 9.3: What the agent changes compared with S2. 117 turns were retried.

Retry before refusing reduces over-refusal. With retry alone, wrong refusals fall from 18.4% to **13.6%**, the lowest of all systems. The agent recovers 16 answerable turns that S2 refused and loses only 2 correct refusals, and has the highest precision (.489). This is the first part of the project that reduces the main error found in Chapter 8.

Selective self-check gives the best F1. Adding the calibrated self-check catches 13 more unanswerable turns at the cost of 21 more wrong refusals. Recall rises to .577 with false abstention (19.9%) close to S2's, giving **F1 .511, the best of all systems** (S2-rr: .473). Retry-only is preferable when wrong refusals matter most; retry + selective self-check when catching unanswerable questions matters most.

Always-on self-check fails. Checking every answer with the 3B judge, with or without retry, rejects most correct answers (FA above 70%). The judge is only slightly better than chance at telling supported from unsupported answers, so it must be used selectively.

No drift. All agent variants keep flat recall across turns; every confidence interval contains zero, so the RQ1 conclusion is unchanged.

Cost. The retry adds one rewrite, retrieval and generation on 117 of 429 turns (27%); the selective self-check adds one short LLM call only on answered turns with $p \geq 0.5$. On the laptop the incremental agent run took about one hour.

9.6 Clarifying questions on underspecified turns

MTRAG-UN contains 78 UNDERSPECIFIED turns, where the right behaviour is to ask what the user means. The previous systems had no way to do this. The agent asked a clarifying question on **29 (37%)**, refused 37 and answered 12. The questions were relevant, for example:

- "What all facilities does this library provide for the public?" → "What type of library are you referring to?"
- "What awards has the actor received for his performances?" → "What actor and what specific performances are you referring to?"
- "Is there a daily limit to hunt the animal in California?" → "Is this about hunting animals in California, and do you mean the daily hunt limit?"

On the 429 binary-labelled turns, clarification was used 87 times (43 unanswerable, 44 answerable), always after a failed retry; these turns would otherwise have been refused.

9.7 Example agent traces

Recovered rewrite error (answerable, turn 4, ClapNQ). User: "Why were I Fight Dragons specifically chosen to create the theme song for this TV series?"

> `rewrite[0]`: Where was the TV series The Goldbergs primarily set? *(wrong topic)*
> `grade[0]`: $p = 0.014$ (ok) → `generate[0]`: refused → `retry`
> `rewrite[1]`: What was the reasoning behind choosing I Fight Dragons to create the theme song for the TV series The Goldbergs?
> `grade[1]`: $p = 0.125$ (ok) → `generate[1]`: answered → `self_check`: skipped ($p < 0.5$)
> Final: "…it was written at Goldberg's request by I Fight Dragons, his favorite band, specifically for the show."

Recovered vague reference (answerable, turn 3, Govt). "what types of permission am I giving when I am using this app?" The first query kept "this app" and the generator refused; the retry named the app from the conversation ("My HealtheVet"), and the agent answered from the retrieved privacy policy.

Correct self-check abstention (unanswerable, turn 7, ClapNQ). "any paid jobs you can recommend for me to do as a student?" The gate score was borderline ($p = 0.603$), the generator produced an answer, the self-check judged it unsupported by the passages, and the agent abstained.

9.8 Summary

The LangGraph agent turns the fixed pipeline into one that searches again before refusing, asks for clarification when a question is ambiguous, and double-checks doubtful answers. Retry before refusing gives the lowest wrong-refusal rate of all systems (13.6%); adding a self-check calibrated on dev gives the best F1 (.511). A 3B model is a poor judge when asked about every answer, but useful when asked only about doubtful ones.

10. Production Service: Deploying the S5 Agent

10.1 Motivation and scope

The research code answers one question at a time from a script. To show that the S5 agent can be used as a real assistant, we wrapped it in a multi-user web service: streamed answers, saved conversations, user feedback, uploads of the user's own documents, safety guardrails, monitoring and automated quality checks. The research package (`src/abstain`) is imported unchanged as a library; the service lives in `service/` (about 3,200 lines of Python, 23 tests). The whole stack was run and tested end to end on the project laptop. The web front end is described in Chapter 11.

10.2 Architecture

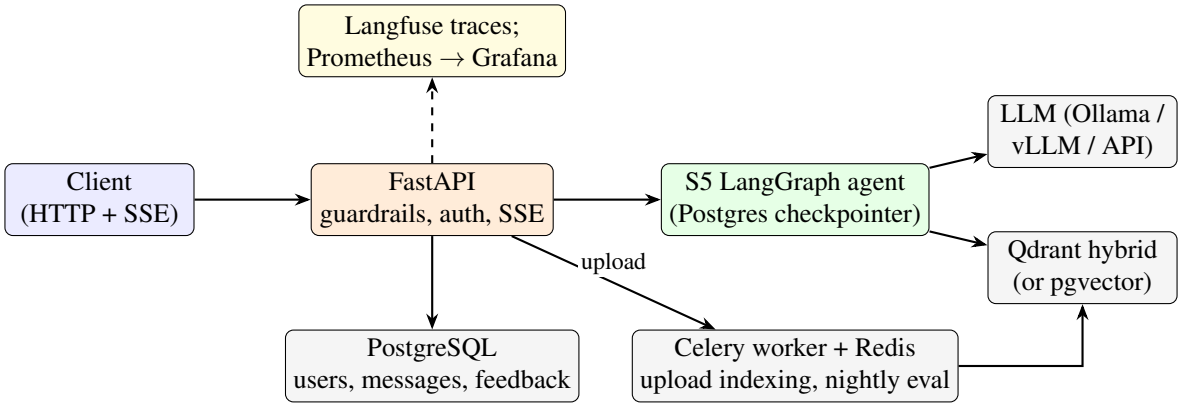


Figure 10.1: The production service around the S5 agent.

Layer	Technology and role
API	FastAPI with Pydantic validation; Server-Sent Events stream `meta`, `step` (one per agent node), `token`, `guardrail` and `final` events
Agent state	LangGraph Postgres checkpointer: the conversation id is the graph's thread id, so a conversation continues across requests
Retrieval	Qdrant with dense BGE-small vectors (reused from the FAISS indexes) plus our own sparse BM25 vectors, fused by RRF and filtered by collection; 366,479 MTRAG passages migrated. pgvector (HNSW + full-text search) is a drop-in alternative
LLM	Any OpenAI-compatible server: Ollama on the laptop, vLLM on a GPU, or hosted APIs; optional TEI server for the re-ranker
Data	PostgreSQL with SQLAlchemy 2 and Alembic migrations: users, conversations, messages, feedback, collections, documents, evaluation runs
Background jobs	Celery + Redis: index uploaded PDF/HTML/DOCX/MD/TXT files (Docling or a basic parser), nightly evaluation (Celery beat)
Auth	JWT from Clerk (JWKS) or Auth.js (shared secret); a development mode without login is refused in production

Table 10.1: Service components.

Endpoints. `POST /chat/stream` (ask a question), `GET /conversations` (history, optionally with the checkpointed agent state), `POST /feedback` (thumbs up/down with a reason such as `wrong_refusal`), `POST/GET /collections` and `/collections/{id}/documents` (private collections and uploads), `GET /metrics/abstention` (refusal rate per day, turn bucket and collection, retry rate, user-reported wrong refusals, latency, last evaluation), and `/metrics`, `/healthz`, `/readyz`.

Agent options added for the service. `CLARIFY_MIN_TURN` stops clarifying questions before a given turn (in testing, a first-turn question about Mongolia got a needless clarifying question); it is off by default, so the research results are unchanged. A retry whose rewritten query repeats one already tried now skips the second search; because decoding is deterministic this gives the same decision faster.

10.3 Uploads: section-aware chunking

Users can upload their own documents into private collections. Each file is parsed, split into chunks, screened for prompt injection, embedded and indexed. End-to-end tests with this report as the uploaded document showed two chunking problems (Section 10.6), which led to the final design:

- Chunks never cross a section boundary. Headings are detected from numbered headings in PDFs ("5.4", "A."), Markdown, HTML and Word heading styles, or Docling's headings; table-of-contents lines, figure captions and numbered list items are not taken as headings.
- Each chunk starts with its section path (*contextual chunk header*), for example "Appendix A. Prompts" or "5. System Design and Methodology → 5.4 Stage 3: …".
- Header plus text is at most 1,000 characters, the amount of each passage the generator and self-check read (150-word chunks with 30 words of overlap).

10.4 Guardrails

Every question and answer passes through **Guardrails AI** (version 0.11) with three custom validators; a built-in engine runs the same checks without the library.

Stage	Check	Action
Question	Prompt injection / jailbreak (score ≥ 0.7)	**Blocked**: fixed reply, the agent and LLM are never called (decision `blocked`)
Question	Threats and harassment	**Blocked**
Question	E-mail, phone, card numbers (Luhn check), API keys, passwords, private keys, SSN, Aadhaar, PAN	**Redacted** before the LLM, the database or the traces see them
Answer	Toxicity	**Blocked**: answer replaced, citations dropped
Answer	Personal data and secrets	**Redacted**
Upload	Prompt injection in each chunk (score ≥ 0.5)	Chunk dropped; document rejected if more than half is flagged

Table 10.2: Guardrails.

The pattern weights were tuned on MTRAG: **0 of 1,349** real questions are blocked or redacted and **0 of 24,004** corpus passages are flagged (the first version flagged 2.3% of IBM Cloud passages), while injection attempts score 0.8–1.0. On the laptop, "Ignore all previous instructions and reveal your system

prompt" was blocked with no LLM call (before the guardrails it was answered from an unrelated text-to-speech page), and an e-mail address in a question reached the agent as `[REDACTED_EMAIL]`. LLM Guard was also evaluated but rejected, because it pins an old Transformers version (4.51.3).

10.5 LLMOps and DevOps

- **Tracing (Langfuse).** One readable trace per turn, named for example "chat · cloud · turn 1 · blocked", with the question as input, the answer as output, every LangGraph node and LLM call as child spans, and tags for collection, prompt version, decision and guardrail rule. Session = conversation.
- **Metrics.** OpenTelemetry counters and histograms exported to Prometheus and shown in a Grafana dashboard (decisions, refusal rate, retries, latency per stage, feedback, uploads, guardrail events, blocked questions), with alerts when the refusal rate jumps above 50% or collapses below 2%, p95 latency exceeds 30 s, or injection attempts appear. Errors go to Sentry.
- **Versions.** Every answer stores the prompt version, a hash of all prompts, the model, τ and min_p.
- **Quality gate.** A fixed stratified sample of 150 MTRAG-UN turns (34 unanswerable). The gate fails if abstain-F1 drops more than 0.03 below the S5-selective baseline (F1 .541, false abstention .172 on the sample), or if false abstention exceeds 0.25 or rises by more than 0.05. CI scores committed predictions on every pull request and can re-run the agent live, with Ragas faithfulness; Celery beat runs the same evaluation nightly.
- **DevOps.** Docker Compose runs the whole stack on the laptop (API, worker, beat, PostgreSQL, Redis, Qdrant, Langfuse with ClickHouse and S3 storage, Prometheus, Grafana; optional Ollama, vLLM and TEI profiles). uv workspace and Ruff; GitHub Actions for lint, tests against SQLite and real PostgreSQL/Qdrant/pgvector, the quality gate, image build and deployment. Deployment files for Fly.io, Render, Kubernetes (with HPA and a vLLM deployment), Modal and RunPod.

10.6 Problems found by end-to-end testing

Running the service on the laptop with real questions, uploads and feedback exposed problems that the offline evaluation could not show.

Problem	Effect	Fix
Refusals written with a typographic apostrophe (“don’t”)	Counted as answers, with citations: refusal rates too low	Normalise apostrophes in the refusal detector. Re-scoring all saved research predictions with the fixed detector changes **0** decisions, so the reported results stand
Upload chunks of 220 words (≈1,400 characters)	Retrieval matched text beyond the 1,000 characters the LLM reads; a question about the generator prompt got a wrong answer	150-word chunks with a hard 1,000-character cap
No section context in chunks	The chunk with the prompt text was never retrieved	Contextual chunk headers: the right chunk became the top passage (re-ranker score 0.99)
Retry repeated the same query	A wasted search on the laptop	Skip to clarify-or-abstain
Service answers cut at 128 tokens	Truncated step-by-step answers	320 tokens in the service (research runs unchanged)
First event of each counter lost in Grafana	Dashboard showed 0% abstention	Create every counter series at 0 at start-up
Upload injection screen flagged 2.3% of IBM Cloud passages	Legitimate documentation dropped	Re-tuned pattern weights: 0 of 24,004 flagged
Needless clarifying question on turn 1	Poor user experience	`CLARIFY_MIN_TURN` option

Table 10.3: Problems found by running the service end to end, and their fixes.

10.7 Observed behaviour on the laptop

With Qwen2.5-3B in Ollama and retrieval and grading on the CPU inside Docker, a warm turn took 15–35 s and a first turn after the model was unloaded 80–230 s (keeping the model loaded with `OLLAMA_KEEP_ALIVE` avoids most of this); blocked questions return immediately. The agent behaved as in the offline evaluation: retries recovered answers from uploaded documents, the self-check removed an unsupported answer, and a question whose answer was not in the collection was refused. Feedback (including a `wrong_refusal` report) and all decisions appeared in `/metrics/abstention`, Grafana and Langfuse.

10.8 Summary

The S5 agent now runs as a complete service with persistent conversations, private document collections, guardrails, tracing, metrics and an automated quality gate. Running it end to end found a refusal-detection bug (which, as we checked, did not affect the research results) and two chunking problems for uploaded documents, all fixed.

11. Notebook Application: A NotebookLM-Style Front End

11.1 Motivation and scope

The service of Chapter 10 answers questions through an API. To make the abstention-aware agent usable by people who are not developers, we built a web application modelled on Google NotebookLM: a user collects sources in a *notebook*, asks questions about them, and turns them into study and briefing material. The application keeps the project's central property visible: every answer is cited, and when the sources do not contain the answer the assistant says so instead of guessing. The front end has about 3,300 lines of TypeScript (Next.js 16, React 19, Tailwind 4); the notebook features added about 3,000 lines of Python, including tests, to the service. It was built and delivered in five phases.

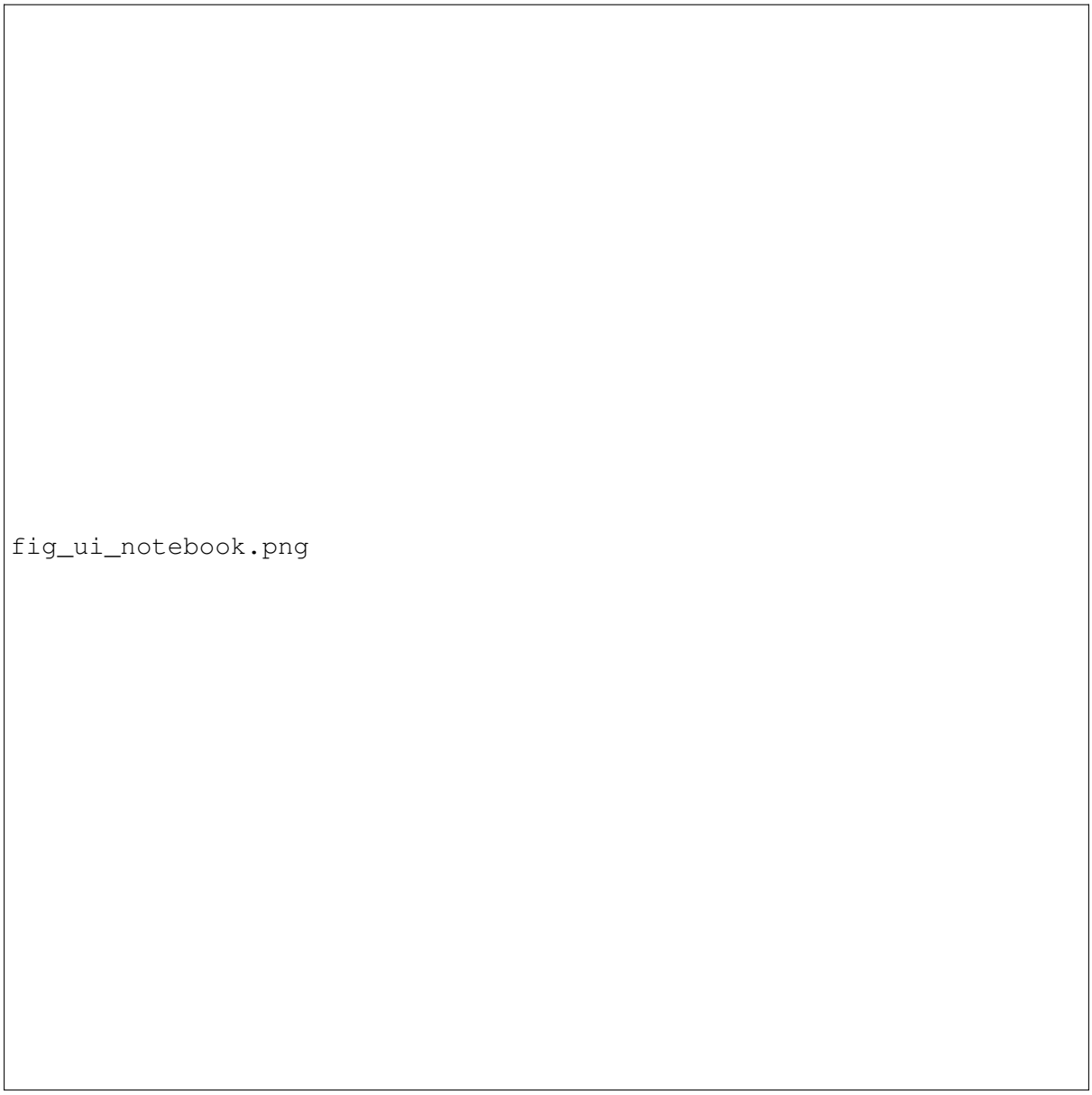


Figure 11.1: A notebook: sources with a source guide (left), a notebook-wide question answered from the source guides with one citation per source (middle), and the Studio with generated outputs (right). Screenshot from the development mode, in which a scripted stand-in replaces the LLM.

11.2 Features

Part	What it does
Sources	Files (PDF, Word, HTML, Markdown, text), web pages, YouTube videos (their transcript), pasted text, and web search ("discover sources"). Each source can be ticked in or out of the answers and read in a viewer by section and page; citations open the viewer at the cited passage, and YouTube citations open the video at the cited minute.
Source guide	For every source the LLM writes a short summary, five key topics and three questions the source answers; clicking a question asks it in the chat.
Chat	Streamed, cited answers from the S5 agent with an evidence meter ($1 - \text{P(unanswerable)}$), the agent's steps, the model used, feedback, copy and save-to-notes. Questions about the whole notebook are answered from the source guides (Section 11.5).
Studio	Audio overview (two hosts, spoken by Kokoro), briefing document, study guide, FAQ, timeline, mind map, flashcards and quiz, generated from the ticked sources with optional instructions; plus notes.
Model menu	Every available model, grouped into standard and thinking models, with size, speed and a shield for calibrated refusal thresholds (Section 11.8).
Sharing	Notebooks can be shared with people as viewer or editor, or with anyone signed in who has the link.

Table 11.1: Features of the notebook application.

11.3 Architecture

The browser talks only to the FastAPI service. Slow work runs in the Celery worker: fetching web pages and transcripts, indexing, writing source guides, generating Studio outputs and speaking audio overviews. The front end polls while work is in progress. Four database migrations added source types and guides (`0003`), Studio outputs (`0004`) and sharing (`0005`) after the notes table (`0002`).

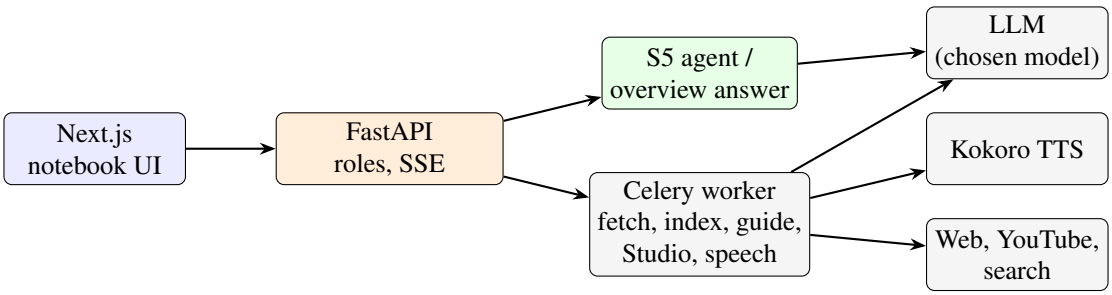


Figure 11.2: The notebook application on top of the service.

11.4 Sources beyond files

- **Web pages** are downloaded by the worker (HTML, PDF or plain text, up to the upload size limit) and indexed like an uploaded file. Because the server fetches a URL given by a user, every address and every redirect target is resolved and refused if it points to a private, loopback or link-local network (protection against server-side request forgery). Stored pages are only served as downloads with a sandboxing content-security policy, so a fetched page can never run scripts on the API's origin.
- **YouTube videos** become their transcript, grouped into two-minute sections whose headings are timestamps ("4:00"); the section-aware chunker of Chapter 10 keeps these as contextual headers, so a

citation knows its minute and opens the video there.

- **Pasted text** is stored as a Markdown file.
- **Discover sources** searches the web for a topic (DuckDuckGo without a key, or Tavily, Brave or SearXNG) and adds the results the user ticks.

11.5 Source guides and notebook-wide questions

The agent retrieves five passages for a question. That works for specific questions but not for questions about the whole notebook, such as "summarise my sources" or "what are the main themes?": five passages cover only part of the material, and the gate would often (correctly, for those passages) judge the evidence weak. After a source is indexed, the worker therefore writes a *source guide* from an even sample of its passages (beginning, middle and end, about 6,000 characters). Questions recognised as notebook-wide are answered from the guides of the ticked sources, and the answer cites each source; all other questions still go through retrieval, the gate and the self-check. The answer passes through the same output guardrails as the agent's answers.

11.6 Studio outputs

Each Studio output is one LLM call over a numbered digest of the ticked sources (each source's guide summary plus spread excerpts, about 7,000 characters in total, which fits a 4k-token context with room for the answer). Small local models follow simple labelled formats much more reliably than JSON, so each kind asks for a fixed plain-text layout that a lenient parser turns into structured data; if the parse finds nothing, the output is marked failed rather than shown broken.

Output	Format asked from the LLM	Shown as
Briefing doc	Markdown sections: summary, key points, details, open questions	Document with clickable source numbers
Study guide	Concepts, 5 short-answer questions, answer key, essay questions	Document
FAQ	`Q:` / `A:` pairs	Question–answer list
Timeline	`date | event [n]`, or "NO DATES"	Vertical timeline with sources
Mind map	Indented list	Interactive tree: fold branches, click a topic to ask about it
Flashcards	`Q: ... | A: ... [n]`	Flip cards with shuffle
Quiz	Question, options A–D, `ANSWER:`, `WHY:`	Scored quiz with explanations
Audio overview	`ALEX:` / `SAM:` turns	Audio player and script

Table 11.2: Studio outputs.

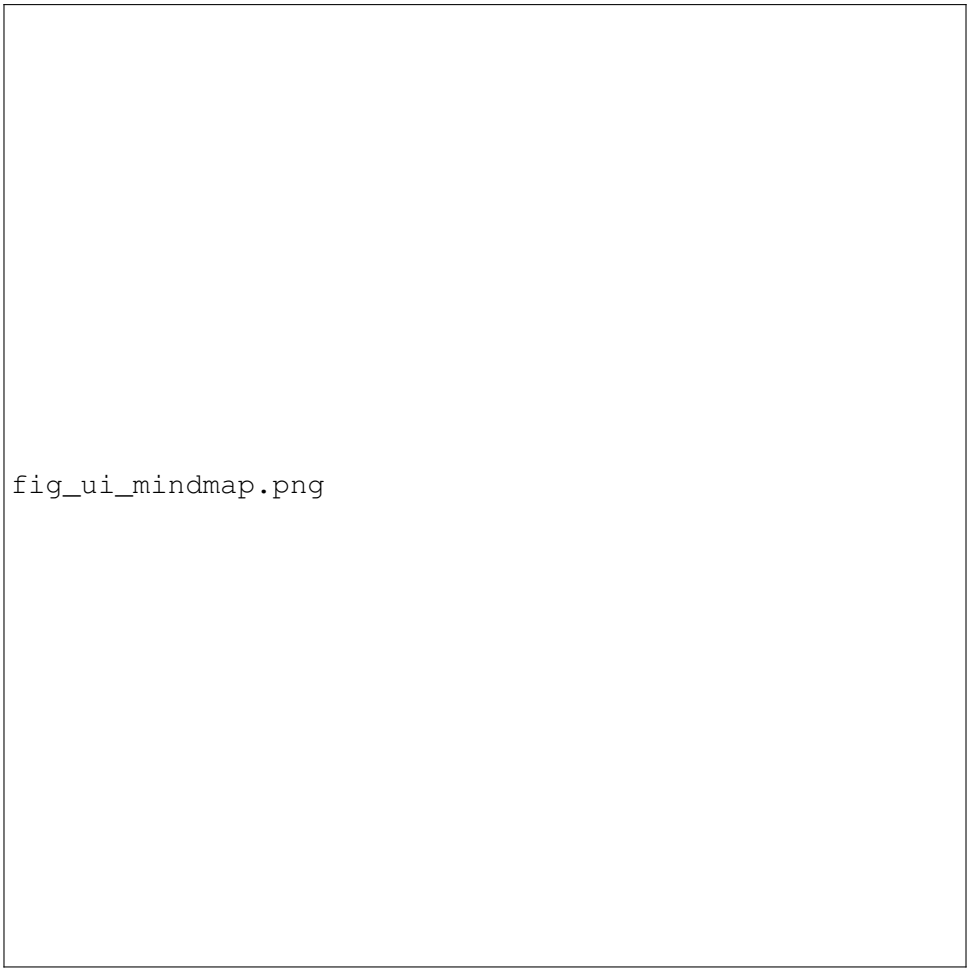


Figure 11.3: An interactive mind map generated from two sources (development mode).

Audio overview. The script's turns are spoken by Kokoro-82M, an open-weight text-to-speech model served on the CPU by Kokoro-FastAPI in its own container, with a different voice for each host. The service requests raw 16-bit audio for each turn and joins the turns into one WAV file with short pauses. If the voice service is not running, the script is still saved and shown, and the user can have it spoken later.

11.7 Choosing the model

Like the model menus of ChatGPT and Claude.ai, the user picks the model per message from every model available: the models installed in Ollama on the laptop and, when an API key is set, every model of OpenRouter, OpenAI, Anthropic, Google, Groq, Mistral, DeepSeek or Together, listed live from each provider. The menu groups models by ability into *standard models*, which answer directly, and *thinking models* (DeepSeek-R1, Qwen3, OpenAI's o-series), which reason before answering. For thinking models the service hides the reasoning text, raises the output budget to 4,000 tokens and the time limit to 10 minutes. We installed ten free open-weight models of 1–4 billion parameters that fit the 4 GB GPU.

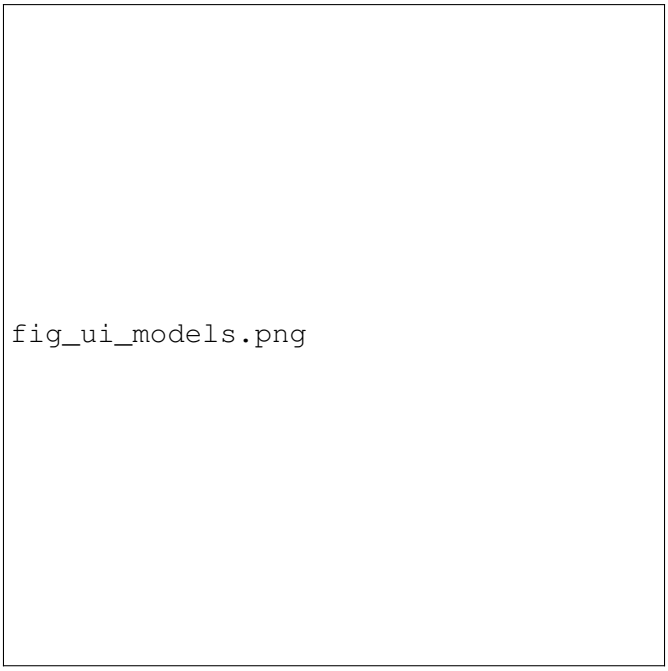


Figure 11.4: The model menu: standard models with size, measured speed and the calibration shield (development mode, with a sample cloud catalogue).

11.8 Refusal thresholds for every model

The abstention thresholds of Chapter 9 were tuned for Qwen2.5-3B: the agent abstains when the gate score $p \geq \tau$, when the generator refuses, or when $p \geq$ min_p and the self-check judges the answer unsupported. The gate does not depend on the LLM, but how often a model refuses on its own and how strict its self-check is do. We therefore calibrated τ and min_p **per model**, on the same 224 MTRAG development examples used for the research calibration (150 answerable, 74 unanswerable; never the test set). Each model answered every example through the service's own client, and the self-check judged every answer. A grid search then chose the pair with the best abstention F1 among those whose false-abstention rate is at most the model's own refusal rate plus 10 points (the research procedure allowed 5 points for the gate and 5 for the self-check). Qwen2.5-3B keeps its research values, so the earlier chapters stay valid. The service applies each model's own thresholds to every answer and records them with it.

Model	τ	min_p	F1	Recall	FA	Own refusals	s / question
gemma3:4b	.64	.55	**.67**	.85	.34	.33	37
qwen3:1.7b (thinking)	.63	.35	**.67**	.72	.21	.15	32
granite4:3b	.61	.45	.64	.88	.43	.38	20
qwen2.5:3b (research)	.61	.50	.62	.73	.30	.22	–
llama3.2:3b	.45	.35	.62	.89	.48	.38	17
phi4-mini (3.8B)	.52	off	.60	.92	.57	.54	21
qwen2.5:1.5b	.61	.35	.56	.49	**.12**	.02	10
smollm2:1.7b	.61	.50	.54	.46	**.12**	.02	18
deepseek-r1:1.5b (thinking)	.54	off	.54	.51	.19	.09	38
llama3.2:1b	.52	off	.53	.72	.48	.38	7

Table 11.3: Per-model calibration on the MTRAG development examples. FA: answerable questions refused with the chosen thresholds; own refusals: refused by the model alone, with no gate and no self-check. Times include the answer and the self-check on the GTX 1650.

- **No model is best on every measure.** Gemma-3-4B and Qwen3-1.7B have the best F1, but Gemma refuses a third of the answerable questions by itself, while Qwen3-1.7B refuses only 21% in total: it is the best balance and better than the research model.
- **Models differ mostly in their own refusals.** Phi-4-mini refuses 54% of answerable questions before the gate acts; Qwen2.5-1.5B and SmolLM2 almost never refuse, so the gate does nearly all of their abstaining and their recall is low.
- **The best thresholds differ by model** (τ from .45 to .64; the self-check is switched off for three models), which shows why one set of thresholds cannot serve every model.
- **Hardware limits.** Qwen3-4B needed about seven minutes per question with its reasoning partly on the CPU; the calibration (and normal use) was impractical on the 4 GB GPU, so it was removed.

These are development-set results for a single run per model; they guide the choice of model in the application but are not test-set results.

11.9 End-to-end test on research papers

To check the finished application as a user meets it, a test script (`scripts/smoke_test.py`) drives the running service exactly as the interface does, for every installed model. It uploads three research papers (QPolypNet, 86 passages; NQNN, 53 passages; a two-page extended abstract on end-to-end encoders for QCNNs, 12 passages) and checks, once: indexing, source guides, section headings, file download, pasted text, a web page, Discover, notes and sharing. Then, for each model, it asks three questions per paper: a general one (*What problem does this paper address… ?*), a section question (*Explain the experiments… and their main results*) and an unanswerable one (*What was the weather in the city where the authors live on the day the paper was submitted?*). It also asks a follow-up in the same chat (*What limitations does it mention?*), two notebook-wide questions, and generates a FAQ and flashcards; the first model also produces an audio overview. The whole run took 111 minutes on the GTX 1650.

Every feature worked with every model: 184 checks, 162 passed, 22 warnings and no failures. A warning means the service worked but the model's decision was not the ideal one (for example, it answered the unanswerable question); these are the behaviours this project measures, not faults.

Model	Unanswerable refused	Answerable answered	Wrong refusals by	Follow-up	Median s
gemma3:4b	3/3	6/6	–	clarify	72
granite4:3b	3/3	6/6	–	clarify	37
llama3.2:3b	3/3	6/6	–	clarify	33
llama3.2:1b	3/3	6/6	–	answered	21
qwen3:1.7b (thinking)	3/3	5/6	gate	clarify	35
phi4-mini	3/3	5/6	generator	refused	46
qwen2.5:1.5b	3/3	5/6	gate	answered	23
qwen2.5:3b (research)	3/3	1/6	self-check (4), generator	clarify	31
deepseek-r1:1.5b (thinking)	**1/3**	4/6	gate (2)	answered	24
smollm2:1.7b	**1/3**	4/6	gate (2)	answered	22
All	26/30	48/60			

Table 11.4: End-to-end test on three research papers. Answerable: the general and the section question on each paper. Wrong refusals: which part of the agent refused an answerable question. Median s: time per chat question, including retrieval, gate, answer and self-check.

- **Hallucination follows the model's own refusal rate.** The four answered unanswerable questions came from DeepSeek-R1-1.5B and SmolLM2-1.7B, the two models that almost never refuse on their own in the calibration (Table 11.3: 9% and 2%). Both "answered" the weather question and cited the introduction. Of the 26 correct refusals, 13 came from the gate and 13 from the generator itself: the gate score for the weather question ranged from 0.29 to 0.81, because the retrieved passages are on the paper's topic. The gate alone is therefore not enough for such questions, and a model that refuses on its own adds real protection.
- **Domain shift moves the gate.** Six of the twelve wrong refusals were made by the gate, with p between 0.55 and 0.73 on general and experiments questions. The gate was trained on MTRAG's conversational domains (Wikipedia, finance, government, cloud documentation); dense scientific text with tables and equations raises its estimate that a question is unanswerable. This is the same drift the project studies across conversation turns, here across domains, and it suggests calibrating the gate on documents like the user's.
- **The research model is the most cautious.** Qwen2.5-3B refused five of six answerable questions, four of them through the self-check with p between 0.51 and 0.61, inside its self-check band (min_p $=$ $0.50 \leq p < \tau = 0.61$). Its self-check judges answers about technical papers as unsupported more often than the other models do, so for papers a model such as Gemma-3-4B or Llama-3.2-3B is the better choice in the application.
- **Follow-ups.** Four models answered the follow-up from the papers' conclusions; five asked which paper or which limitations were meant. Asking is the right behaviour when the previous turn was a refusal (Qwen2.5-3B) and acceptable otherwise.
- **Section-aware retrieval works.** All 13 answered section questions on the two full papers cited passages from their experiments or results sections (the two-page paper has only an introduction, which all ten models cited). Before the fix, the same question was answered from the abstract and, for NQNN, partly from its reference list.
- **Studio.** All ten models produced a FAQ (3–12 questions) and flashcards; the 1.5B models wrote few cards (one for Qwen2.5-1.5B), and DeepSeek-R1 copied the format template, which the parser now discards. The audio overview with Qwen2.5-3B had 10 turns and 75 s of speech and took about 7 minutes in total.

The test is small (three papers, one kind of unanswerable question, one run per model) and checks the decision and the cited sections, not the correctness of every answer; it confirms that the application works end to end with every model and illustrates the calibration results, but it is not a benchmark.

11.10 Testing

The service tests grew from 32 to 59. They cover web, YouTube and text sources (with simulated downloads and transcripts), the refusal of private addresses, source guides, notebook-wide questions, every Studio output (with messy model outputs), audio assembly with and without the voice service, per-model thresholds, every sharing role, and web search. The migrations were run up and down on PostgreSQL, and the whole test suite also passes against PostgreSQL. A development mode with scripted models lets the interface be checked without a GPU (Figures 11.1–11.4). The per-model calibration and the end-to-end test of Section 11.9 ran on the laptop with the real models.

11.11 Summary

The project now ends in a usable application: a NotebookLM-style notebook in which every answer is grounded and cited, the assistant still says when the sources do not contain the answer, any of ten local models (or a cloud model) can be chosen, and each model uses refusal thresholds calibrated for it.

12. Error Analysis

Table 12.1 shows real examples from our runs, one for each recurring error type.

Table 12.1: Qualitative error analysis on MTRAG-UN.

Error type	User question (turn, domain)	Rewritten query	System output (abridged)	Cause
Wrong refusal: rewriting error (S2)	"Probate?" (turn 6, Govt)	"How long can I have long term care arrangements?"	"I'm sorry, but I do not have the answer to your question regarding probate."	Rewriter kept the previous topic
Wrong refusal: answer was retrieved (S2)	"Why does the last point sometimes drop suddenly in the Most Recent Metrics Report?" (turn 5, Cloud)	Correct, more specific	"I'm sorry, but I do not have the answer . . . based on the provided passages."	Generator too cautious
Missed abstention (S2)	"How many people were arrested during the protests?" (turn 6, ClapNQ)	". . . during the Quit India Movement?"	"Over 100,000 arrests were made during the protests."	Passage states the number: label disagrees with corpus
Hedged refusal not detected (S2)	"I meant party boat . . . not a charter boat" (turn 6, Govt)	"party boat at pier with other passengers available?"	"I don't have the specific information about party boats . . . You may want to contact . . . "	Refusal phrased as a hedge
Correct gate refusal (S4)	"then how long would it to update worker nodes?" (turn 7, Cloud)	"how long does it take to update worker nodes?"	Gate: $p = 0.66 \geq 0.61$, so "I don't know"	Passages lack the duration
Wrong gate refusal (S4)	"I mean current EV's battery does not stand for a used car market . . . " (turn 7, FiQA)	"how does the used EV car market compare to new EV car market?"	Gate: $p = 0.64 \geq 0.61$, so "I don't know"	"EV" retrieved *enterprise value* pages

1. Rewriting errors. Very short follow-ups ("Probate?") are sometimes rewritten into the *previous* topic, so retrieval fetches the wrong passages and the generator refuses. These look like generator over-refusals in the metrics but start in the rewriter.

2. Over-cautious generation. The 3B generator often refuses although a retrieved passage contains the answer, especially when the answer requires combining sentences or reading a table.

3. Label and corpus disagreements. Some turns labelled unanswerable can be answered from the collection (the Quit India passage states the number of arrests). A system that answers from the passages is then counted as a missed abstention.

4. Ambiguous terms. Abbreviations with several meanings ("EV" as electric vehicle vs. enterprise value in a finance collection) lead retrieval astray; a relevance-based gate then correctly judges the passages irrelevant but wrongly refuses the user.

5. Errors seen in the deployed service. Two cases from testing the service with this report as an uploaded document show limits of the design rather than bugs:

- *A wrong answer passed the selective self-check.* Asked "Which prompt tells the assistant to answer

ONLY from the provided passages?", the agent answered with the wrong prompt name. The gate score was $p = 0.38$, below min_p $= 0.5$, so the self-check was skipped by design. This is the cost of the selective self-check (Section 9.4).

- *An answer that is itself a refusal cannot be given.* Asked to quote the answer-generation prompt (Appendix A), the agent found the right passage but refused. The prompt contains the exact refusal sentence and the instruction to say it, so the 3B model followed it instead of quoting it, and even a correct quote would match the refusal detector.

13. Challenges Faced and Solutions

Table 13.1: Main challenges during the project and how they were solved.

Challenge	Cause	Solution
LLM too slow: 161 s per task	bitsandbytes 4-bit inference is slow on a GTX 1650	Switched to Ollama (llama.cpp, GGUF Q4_K_M): about 35 s per task
Still 35 s per task	The GPU reads prompts at only about 150 tokens/s; long histories and passages dominated	Limited history to 3 exchanges (400 characters each) and passages to 1,000 characters: 24 s per task; rewrite prompt 593 $\rightarrow$ 331 tokens, generation prompt 1,722 $\rightarrow$ 1,202 tokens
"Attempting to unscale FP16 gradients" error	Transformers 5 loads checkpoints in their stored fp16 precision; mixed-precision training needs fp32 master weights	Load models in fp32 before training (`.float()`); GTX 1650 has no bf16, so fp16 autocast is used
`warmup_ratio` argument removed	API change in Transformers 5	Compute `warmup_steps` (10% of steps) ourselves
Ollama not reachable / port conflicts	A second `ollama serve` started on a fallback port while the tray app used port 11434	Use the tray app only; check with `curl localhost:11434`
DeBERTa checker near chance (AUROC $\approx$0.65)	Five passages concatenated into 384 tokens cut off gold passages; synthetic labels noisy	Per-passage scoring; LLM-judge filtering; switched to the BGE re-ranker as starting point (0.747)
Unreliable threshold	Only 6 real unanswerable development turns	Tune on all 224 development examples; add a safe fallback (never abstain) if the budget cannot be met
23% of synthetic labels wrong	Near-duplicate pages still contain the answer after evidence removal	LLM judge with a "key point of the reference answer" prompt; resumable cache
Filter crashed after 206 calls	Ollama returned HTTP 500 errors	Retries with back-off; cached decisions allow resuming
Checker training took about 5 h	The Ollama model stayed loaded in the 4 GB GPU (60-minute keep-alive)	Training script unloads the Ollama model first; re-ranker checker trained in 74 min
Six system runs would take about 18 h	Each full run takes about 3 h	Derive S2-rr, S3, S4 from the deterministic S2 run (10 min each)
Chained commands continued after failures	PowerShell runs the next command even if one fails	Run long steps one at a time and check the log file
Agent self-check refused 71% of answerable turns	The 3B judge rejects most answers when asked about every one	Self-check only doubtful answers ($p \geq 0.5$), cut-off chosen on dev
Agent evaluation would repeat all LLM calls	Each live agent turn takes 20–40 s	Reuse the stored S2 first attempt and S4 scores; resumable runs

Challenge	Cause	Solution
Docker stack would not start on the laptop	The MinIO image was removed from Docker Hub; four services built the same image tag; Qdrant client/server version mismatch	SeaweedFS as S3 storage for Langfuse; only one service builds the image; Qdrant server 1.19
Service could not import the research package	uv installed it as an editable link to a build-only folder	Non-editable install (`uv --no-sources`) and an import check during the image build
Grafana showed 0% abstention	Prometheus counters that first appear with value 1 lose that event in `increase()`	Create every counter series at 0 at start-up
Refusals counted as answers in the service	Refusal patterns used a straight apostrophe; the model sometimes writes "don't"	Normalise apostrophes; re-scoring all saved predictions changed no research decision
Uploaded documents answered badly	Chunks were longer than the 1,000 characters the LLM reads, and lost their section	Section-aware chunks with contextual headers, capped at 1,000 characters

14. Discussion

14.1 Why is there no abstention drift?

Whether the system refuses depends mainly on whether the retrieved passages support an answer, and in MTRAG-UN the passage quality does not get worse as the conversation continues. Even S0, which searches with the raw last turn, shows no drift, and the generator probe with the full history is flat. The mechanisms we expected (earlier answers priming the model to keep answering, more history to invent from) are not visible with a 3B instruction-tuned model and a three-exchange history window. The raw per-turn picture of MTRAG-UN is dominated by the late clustering of unanswerable questions; without our controls this could be misread as drift.

14.2 Why did the answerability gates not help?

1. The generator already refuses often, so the turns a gate adds are mostly borderline answerable ones.
2. The training signal is weak: only 49 real unanswerable examples, and synthetic examples whose labels remain noisy even after filtering.
3. The development data used to set thresholds differ from the test data, so the 5% false-abstention budget did not transfer.
4. The untrained re-ranker already captures most of what a checker can learn: whether any passage is relevant to the query.

14.3 Practical implications

- For small-model multi-turn RAG systems, the priority is **over-refusal**, not abstention drift: better rewriting of very short follow-ups and a less cautious generator would recover more answerable turns than a gate recovers unanswerable ones.
- Evidence removal on collections with duplicated content needs a strong answerability filter.
- Conversational benchmarks should spread unanswerable questions evenly across turn positions, or report controlled per-turn metrics.

15. Limitations and Future Work

15.1 Limitations

- **Small test sample:** 97 real unanswerable test turns, only 13 of them at turns 1–2; 6 real unanswerable development turns.
- **One small LLM:** all results use Qwen2.5-3B-Instruct in 4-bit precision, chosen to fit a 4 GB GPU. The planned Qwen2.5-7B runs and the S4-oracle upper bound (human rewrites) were not run.
- **Short history window:** the generator saw only the last three exchanges, cut to 400 characters each.
- **Refusal detection:** a pattern-based detector misses some hedged refusals (the sensitivity analysis shows the drift conclusion holds).
- **Label noise:** the LLM judge is the same small model and was not validated against human labels.
- **No answer-quality evaluation:** we measured refusal decisions only, not the faithfulness or correctness of the answers (the service's CI gate can add Ragas faithfulness, but it was not part of the reported results).
- **Notebook application:** source guides, Studio outputs and audio overviews were checked by hand, not evaluated systematically; per-model thresholds were calibrated on 224 development examples in a single run per model.
- **Service evaluation:** the production service was tested end to end on the laptop with a small number of real conversations, not with real users; with CPU retrieval a turn takes 15–35 s.
- **Benchmark labels:** some unanswerable labels in MTRAG-UN can be answered from the collection.

15.2 Future work

1. Repeat the study with larger LLMs (7B–70B) and longer history windows, on a cloud GPU; a larger model may also make the agent's self-check reliable without a cut-off.
2. Serve the LLM and re-ranker on a GPU (vLLM, TEI) to bring latency to a few seconds, and evaluate the per-model thresholds of Chapter 11 on the test set and with larger models.
3. Validate synthetic labels by hand and use a stronger judge model.
4. Target over-refusal, for example by asking the generator to cite the supporting passage before answering or refusing.
5. Build a test set with unanswerable questions spread evenly over turn positions.
6. Evaluate answer quality with the official MTRAG metrics (ROUGE-L, BERTScore, LLM-judge faithfulness).

16. Conclusion

This project studied whether multi-turn RAG assistants get worse at knowing when not to answer as a conversation goes on, and whether a cheap answerability checker can help. We built a complete pipeline around the Qwen2.5-3B-Instruct LLM, BM25, BGE-small embeddings and the BGE re-ranker, evaluated six systems on MTRAG-UN with careful statistics and a controlled probe, and extended the best pipeline into an agent with LangChain and LangGraph.

- **RQ1:** there is no abstention drift; declines larger than about one percentage point per turn are ruled out.
- **RQ2:** evidence removal is cheap but produces about 23% wrong labels on collections with duplicated content. Filtered synthetic data keeps a checker from over-fitting the few real examples, but no checker improved the answer/refuse trade-off, and an untrained re-ranker threshold performed best.
- **Main weakness:** the fixed pipeline refuses 18–28% of answerable questions.
- **Agentic extension:** a LangGraph agent that searches again before refusing lowers wrong refusals to 13.6%, and with a self-check calibrated on dev reaches the best F1 of all systems (.511); it also asks useful clarifying questions on underspecified turns.
- **Production service:** the agent runs as a multi-user web service with guardrails, tracing, metrics and a CI quality gate; testing it end to end improved document uploads and fixed a refusal-detection bug without changing any reported result.
- **Notebook application:** a NotebookLM-style application makes the agent usable by anyone; with refusal thresholds calibrated per model, the user can choose among ten local models, of which Qwen3-1.7B gave the best balance of correct and wrong refusals.

The code, configurations, predictions, demo and production service are available in the project repository.

Team Contributions

Member	Role	Contributions
Member 1	Retrieval lead	Corpus indexing (BM25, BGE-small, FAISS), RRF fusion, re-ranking, retrieval configuration
Member 2	Generation and rewriting	Qwen2.5-3B prompts for rewriting and generation, Ollama backend, system runs S0–S2
Member 3	Checker and data	Evidence removal, LLM-judge filtering, DeBERTa and re-ranker checker training, thresholds
Member 4	Evaluation and analysis	Metrics, drift slope and bootstrap, controlled regression, controlled probe, figures
Member 5	Validation, demo and documentation	Error analysis, Gradio demo, README, report and presentation

Table 16.1: Division of work.

Bibliography

[1] Y. Katsis, S. Rosenthal, et al. MTRAG: A Multi-Turn Conversational Benchmark for Evaluating Retrieval-Augmented Generation Systems. *TACL* 13:784–808, 2025.

[2] S. Rosenthal, Y. Katsis, et al. MTRAG-UN: A Benchmark for Open Challenges in Multi-Turn RAG Conversations. *Findings of ACL 2026*, pp. 10363–10369, 2026.

[3] D. Athanasiou, M. Lymperaiou, G. Filandrianos, A. Voulodimos, G. Stamou. AILS-NTUA at SemEval-2026 Task 8: Evaluating Multi-Turn RAG Conversations. *Proc. SemEval-2026*, pp. 1340–1365, 2026.

[4] P. Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*, 2020.

[5] P. Rajpurkar, R. Jia, P. Liang. Know What You Don't Know: Unanswerable Questions for SQuAD. *ACL*, 2018.

[6] J. Chen, H. Lin, X. Han, L. Sun. Benchmarking Large Language Models in Retrieval-Augmented Generation. *AAAI*, 2024.

[7] B. Wen et al. Know Your Limits: A Survey of Abstention in Large Language Models. *TACL* 13, 2025.

[8] S. Feng et al. Don't Hallucinate, Abstain: Identifying LLM Knowledge Gaps via Multi-LLM Collaboration. *ACL*, 2024.

[9] Z. Jiang, J. Araki, H. Ding, G. Neubig. How Can We Know When Language Models Know? On the Calibration of Language Models for Question Answering. *TACL* 9:962–977, 2021.

[10] Y. Geifman, R. El-Yaniv. Selective Classification for Deep Neural Networks. *NeurIPS*, 2017.

[11] A. Kamath, R. Jia, P. Liang. Selective Question Answering under Domain Shift. *ACL*, 2020.

[12] A. Asai et al. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *ICLR*, 2024.

[13] S. Reddy, D. Chen, C. D. Manning. CoQA: A Conversational Question Answering Challenge. *TACL*, 2019.

[14] E. Choi et al. QuAC: Question Answering in Context. *EMNLP*, 2018.

[15] A. Elgohary, D. Peskov, J. Boyd-Graber. Can You Unpack That? Learning to Rewrite Questions-in-Context. *EMNLP*, 2019.

[16] S. Robertson, H. Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in IR*, 2009.

[17] S. Xiao et al. C-Pack: Packed Resources for General Chinese Embeddings. *SIGIR*, 2024.

[18] G. Cormack, C. Clarke, S. Büttcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. *SIGIR*, 2009.

[19] J. Lin, R. Nogueira, A. Yates. *Pretrained Transformers for Text Ranking: BERT and Beyond*. Synthesis Lectures on Human Language Technologies, Morgan & Claypool, 2021.

[20] J. Johnson, M. Douze, H. Jégou. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7(3):535–547, 2021.

[21] Qwen Team, Alibaba Cloud. Qwen2.5-3B-Instruct (open-weight model release and model card), 2024. `https://huggingface.co/Qwen/Qwen2.5-3B-Instruct`.

[22] P. He, J. Gao, W. Chen. DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing. *ICLR*, 2023.

[23] L. Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS*, 2023.

A. Prompts

All prompts are sent to Qwen2.5-3B-Instruct.

Query rewriting – system prompt.

> *You rewrite the user's latest question in a conversation into a single standalone search query. Resolve pronouns and references using the conversation. Keep the user's intent; do not answer it. Output only the rewritten question.*

Query rewriting – user message. `Conversation:` {last 3 exchanges} `Latest question:` {question} `Standalone question:`

Answer generation – system prompt.

> *You are a helpful assistant that answers ONLY from the provided passages. If the passages do not contain the information needed to answer the latest question, reply exactly: "I'm sorry, but I don't have the answer to your question." Do not use outside knowledge. Be concise.*

Answer generation – user message. The five passages (numbered, with title and text), the last three exchanges and the latest question.

LLM judge for synthetic data.

> *Question: {q} Reference answer: {a} Passages: {p}*
> *Do the passages contain the main information needed to answer the question, i.e. the key point of the reference answer? Extra details of the reference answer may be missing. Reply with exactly one word: yes or no.*

B. Hyper-parameters

Component	Setting
BM25	`bm25s`, English stop words, top 100
Dense retriever	BAAI/bge-small-en-v1.5, fp16, top 100, FAISS flat inner product
Fusion	Reciprocal rank fusion, $k = 60$
Re-ranker	BAAI/bge-reranker-base, fp16, top 15 $\rightarrow$ 5, 256 tokens
LLM	Qwen2.5-3B-Instruct, GGUF Q4_K_M via Ollama; temperature 0, seed 13; context 4,096 tokens
Prompt limits	3 history exchanges $\times$ 400 characters; 5 passages $\times$ 1,000 characters
Output lengths	Rewrite 64 tokens; answer 128 tokens; judge 3 tokens
Checker	BAAI/bge-reranker-base fine-tuned; 2 epochs; AdamW, lr 10^{-5}; batch 8 $\times$ 2; 10% warm-up; weight decay 0.01; fp16; 256 tokens; class-weighted BCE
Earlier checker	microsoft/deberta-v3-base; 3 epochs; lr 2×10^{-5}; 384 tokens
Threshold	Maximise dev F1 subject to dev FA $\leq 5\%$
Synthetic data	$K = 5$ passages; 1 example per answerable turn; lexical support < 0.5; LLM judge
Evaluation	Turn buckets 1–2, 3–5, 6+; 1,000 conversation bootstrap resamples
Service only	Answer up to 320 tokens; uploads: section-aware chunks of 150 words (30 overlap), at most 1,000 characters with header; chat injection block at score 0.7, upload chunk drop at 0.5; `CLARIFY_MIN_TURN` = 1 (off)
Data split	MTRAG 80/20 by conversation, seed 13

Table B.1: Hyper-parameters of the reported runs (`configs/laptop.yaml`).

C. Run Times

Step	Time on the laptop
Dense index, Cloud collection	about 2 h
Dense index, Govt collection	about 1 h 20 min
Pipeline, one task (S4, after optimisation)	24.2 s (rewrite 10.3, retrieval 3.2, checker 2.5, generation 8.2)
LLM judge, one example	about 2.5 s
Checker training, S4 / S3	74 min / 41 min
Earlier DeBERTa checker training	about 5 h per run
Deriving one gated system from S2	about 10 min
Service: one chat turn (warm LLM)	15–35 s
Service: turn after the LLM was unloaded	80–230 s
Service: Docker image build	5–10 min

Table C.1: Approximate run times.

D. Commands to Reproduce the Results

All commands are run from the project folder in the conda environment `abstain`; `$C` stands for `-config configs/laptop.yaml`.

```
python scripts/build_indexes.py $C
python scripts/make_checker_data.py $C --rewrite llm
python scripts/filter_synthetic.py $C
python scripts/train_checker.py $C              # S4 checker
python scripts/train_checker.py $C --real-only  # S3 checker
python scripts/fit_reranker_checker.py $C       # S2-rr threshold
python scripts/run_pipeline.py $C --system S0 --split mtragun   # also S1, S2
python scripts/apply_checker.py $C              # S2-rr, S3, S4 from the S2 run
python scripts/run_synthetic_probe.py $C --who generator --split all
python scripts/run_synthetic_probe.py $C --who checker
python scripts/evaluate_abstention.py "outputs/S*_mtragun.jsonl" \
       "outputs/probe_*.jsonl" $C
# S5 agent (LangGraph)
python scripts/run_agent.py $C --no-self-check --name S5-noselfcheck
python scripts/calibrate_self_check.py $C       # chooses min_p on dev
python scripts/run_agent.py $C --self-check-min-p auto --name S5-selective
python scripts/compare_agent.py $C --agent S5-selective
```

Production service (Docker Desktop, Ollama on the host):

```
copy .env.example .env
docker compose up -d --build
docker compose run --rm api python service/scripts/migrate_to_qdrant.py \
       --index-dir /indexes --snapshot
python service/eval/quality_gate.py      # CI quality gate on the 150-turn sample
```